EARTHQUAKE DAMAGE ASSESSMENT AND DETERMINATION OF ASSISTANCE NEEDS VIA SOCIAL MEDIA CHANNELS

DILARA HALAVURT

ÖZYEĞİN UNIVERSITY

FEBRUARY 2026

EARTHQUAKE DAMAGE ASSESSMENT AND DETERMINATION OF ASSISTANCE NEEDS VIA SOCIAL MEDIA CHANNELS

by
Dilara Halavurt

Thesis
Submitted in Partial Fulfillment of the
Requirements for the Degree of

Master of Science

in
Computer Science

Advisor: Prof. Olcay Taner Yıldız

Graduate School of Science and Engineering
Özyeğin University
İstanbul

February 2026

EARTHQUAKE DAMAGE ASSESSMENT AND DETERMINATION OF ASSISTANCE NEEDS VIA SOCIAL MEDIA CHANNELS

Approved by:

Prof. Olcay Taner Yıldız, Advisor
Department of Computer Science
Özyeğin University

Asst. Prof. Zeynep İlknur Karadeniz Erol
Department of Artificial Intelligence and Data Engineering
Özyeğin University

Prof. Arzucan Özgür Türkmen
Department of Computer Engineering
Boğaziçi University

Approval Date: February 2026

DECLARATION OF ORIGINALITY

I hereby declare that I am the sole author of this thesis and that this is the true copy of my thesis, including the final revisions, approved by my thesis committee. All data and information have been obtained, produced, and presented in accordance with the rules of research ethics and principles of academic honesty. As required by these rules, to the best of my knowledge I have acknowledged ideas, thoughts, and any copyrighted material in accordance with the standard referencing rules. I certify that any part of this thesis has not been submitted for a degree or diploma in another educational institution.

Dilara Halavurt

ABSTRACT

This thesis develops an end-to-end system for automated earthquake damage assessment using social media data from Ekşi Sözlük. The system includes a cross-platform emergency reporting application, a data collection pipeline that gathered 24,900 entries from six major Turkish earthquakes, and a preprocessing module for Turkish text normalization. A multi-stage annotation scheme labels sentences for damage, assistance needs, and information content, with a secondary stage for severity classification. Four fine-tuned BERT classifiers achieved accuracies of 97.52%, 95.15%, 95.20%, and 90.42% for these respective tasks, outperforming TF-IDF and zero-shot LLM baselines. The models were deployed as a containerized AWS Lambda service, demonstrating that social media can provide actionable intelligence for disaster response.

Keywords: Social Media, Disaster Management, Natural Language Processing, BERT, Earthquake Damage Assessment

ÖZET

Bu tez, Ekşi Sözlük sosyal medya verilerini kullanarak otomatik deprem hasar değerlendirmesi için uçtan uca bir sistem geliştirmektedir. Sistem; platformlar arası acil durum raporlama uygulaması, altı büyük Türkiye depreminden 24.900 girdi toplayan veri hattı ve Türkçe metin normalleştirme modülünden oluşmaktadır. Çok aşamalı etiketleme şeması, cümleleri hasar, yardım ihtiyacı ve bilgi içeriği açısından sınıflandırmakta; ikinci aşamada ise hasar şiddeti belirlenmektedir. Dört adet ince ayarlı BERT sınıflandırıcısı, bu görevlerde sırasıyla %97,52, %95,15, %95,20 ve %90,42 doğruluk elde ederek TF-IDF ve sıfır atışlı büyük dil modeli yaklaşımlarını geride bırakmıştır. Modeller konteynerleştirilmiş AWS Lambda hizmeti olarak dağıtılmış olup sosyal medyanın afet müdahalesinde eyleme dönüştürülebilir bilgi sağlayabileceği gösterilmiştir.

Anahtar Kelimeler: Sosyal Medya, Afet Yönetimi, Doğal Dil İşleme, BERT, Deprem Hasar Değerlendirmesi

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my advisor, Prof. Olcay Taner Yıldız, for his guidance and support throughout this research. His expertise in natural language processing and his encouragement were invaluable in completing this thesis.

I am deeply grateful to my company, Softtech, for their tremendous support throughout the writing of this thesis. Their resources, flexibility, and encouragement made it possible for me to pursue this research while maintaining my professional responsibilities.

I am also grateful to my family for their unwavering support and understanding during my graduate studies.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ACRONYMS AND ABBREVIATIONS

AI Artificial Intelligence

AIDR Artificial Intelligence for Disaster Response

API Application Programming Interface

ASCII American Standard Code for Information Interchange

AWS Amazon Web Services

BERT Bidirectional Encoder Representations from Transformers

CORS Cross-Origin Resource Sharing

CPU Central Processing Unit

CSV Comma-Separated Values

ECR Elastic Container Registry

HTML HyperText Markup Language

HTTP HyperText Transfer Protocol

HTTPS HyperText Transfer Protocol Secure

ICT Information and Communication Technology

JSON JavaScript Object Notation

LLM Large Language Model

NER Named Entity Recognition

NLP Natural Language Processing

PIO Public Information Officer

POS Part of Speech

SDK Software Development Kit

SVM Support Vector Machine

TF-IDF Term Frequency-Inverse Document Frequency

UGC User-Generated Content

URL Uniform Resource Locator

UUID Universally Unique Identifier

1. INTRODUCTION

1.1 Background and Context

The dangers that earthquakes pose to society are many and varied. In addition to the risk of loss of life and property damage and loss of buildings and other physical infrastructure, there may also be disruptions in daily life and in the economy and in society as a whole. Since Turkey is located along many different major fault lines it is likely to experience some of the most severe and frequent earthquakes in the world; therefore, there is a need to develop disaster response systems to help mitigate the effects of earthquakes.

Over the past couple of decades, the way in which people communicate about emergency situations has undergone significant changes due to the increasing use of social media. Users can now use platforms like Twitter and Ekşi Sözlük to instantly publish information concerning emergencies; as a result, they can create a tremendous amount of relevant information that could be useful in managing disasters. However, many current systems that have been created to assess the damage caused by earthquakes and to coordinate disaster response activities are unable to efficiently utilize the social media data generated during an earthquake. This is largely due to the fact that social media data contains "noise," lacks enough context, and is usually provided in a disorganized manner.

As a result of the high costs associated with obtaining full access to all data available on Twitter, the original intention of this project to obtain data from both Twitter and Ekşi Sözlük was not possible, based on the constraints of this project. Therefore, this project will focus on gathering data from Ekşi Sözlük, as this site has the ability to provide valuable information to determine the success of disaster response systems and fits within the budget of this project.

This study will evaluate the utilization of Ekşi Sözlük data in disaster management systems, by identifying the main barriers to extracting and using real-time data and

analyzing data related to earthquake responses.

1.2 Problem Statement

Technological developments in emergency response and disaster management have advanced significantly in recent years. The process of assessing damage after earthquakes typically utilizes one of two types of methods: either collecting data through surveying, walking and/or using sensor networks and/or satellite remote sensing; both of which are generally time-consuming, geographically limited, and costly.

On the other hand, social media platforms are an effective alternative method that may be used as a source for gathering user-generated content UGC, due to their ability to gather large amounts of data in great detail over a short period of time. Nevertheless, the social media aspect of emergency response and disaster management systems currently suffers from a number of significant problems when it comes to processing social media data:

1. Social media data is usually unorganized, noisy, and available in many different languages and formats.

2. It is difficult to transform the social media data into actionable information in near-real-time concerning what geographic areas were most impacted and what services or resources will be required.

3. There is also a lack of integration between social media data and existing disaster management processes.

Thus, this research will utilize Ekşi Sözlük data for cross-platform social media applications and develop new NLP and AI techniques to assist with addressing the above-mentioned problems.

1.3 Motivation

More frequent and severe earthquakes are happening now in Turkey than at any other time in history, and as such most of the methods historically used to respond to earthquakes are no longer effective. In effect, this is providing an opportunity to develop new methods to address earthquake-related disasters. Collecting data generated by the public from real-time social media postings can be utilized to provide more timely ways to respond to disasters; however, there is little evidence that responders are utilizing this capability in their responses to disasters. As such, completing this project will help us not only to improve how disaster responses occur but also to contribute to the larger body of work currently being done to incorporate digital technology into emergency planning and response.

There are four primary motivations to accomplish this study:

- The increasing demand to quickly assess damage caused by a natural disaster after the occurrence of a disaster.
- The increased utilization of social media data to document the occurrence of disasters.
- The continued development of new AI technologies to process large volumes of unorganized data.
- The possible enhanced emergency response times and resource distribution with AI technology.

1.4 Research Objectives

The four main objectives of this thesis are:

1. **Creating the application:** An application that allows users to submit disaster reports via a computer or mobile device immediately after an earthquake occurs, by allowing them to provide their location; the extent of damage to buildings; and any immediate needs they may have.

2. **Corpus construction and preprocessing:** We plan to collect and process user-generated content about earthquakes from Ekşi Sözlük, and develop a processing pipeline to convert raw, ungrammatical, and noisy social media content into a format that is suitable for machine learning algorithms. The processing pipeline will need to handle morphological properties of the Turkish language; correct spelling mistakes; and convert the text from ASCII to Unicode.

3. **Developing a classification model:** We will develop, test and evaluate various classification models using traditional machine learning techniques; transformer architecture; and prompt-based models of large language models to categorize user-generated content as either reporting physical damage (H vs. NH); asking for help (Y vs. NY); providing factual information (B vs. NB); and reporting damage severity (mild vs. severe), where applicable, to those reports identified as damage-related. The goal is to determine which of these techniques will be most suitable for use in real-time operational deployment.

4. **Creating an end-to-end system:** Using the most effective models developed in the above objective, we will design an end-to-end system that uses a cloud-based inference service to receive new user-generated content in real-time, classify the content in all categories, and direct damage-related content through the damage severity classifier. The system will also be designed to be scalable during periods of peak demand immediately following an earthquake.

Initially, there were plans to incorporate a named entity recognition NER module

into the overall research project; however, due to limitations imposed by the constraints of the dissertation, the decision was made to leave the NER module as future work. Thus, the current preprocessing pipeline serves as a foundation for what will eventually be a NER module.

1.5 Structure of Thesis

This dissertation will be divided as follows:

- **Chapter 2: Literature Review** — This chapter includes reviews of the prior literature on disaster management, social media analytics and natural language processing (NLP). It will identify gaps in previous literature and position this research within the larger body of knowledge.
- **Chapter 3: Methodology** — This chapter will detail how the data was collected, preprocessed, and annotated, and how the models were trained to support the project. The tools and frameworks used to develop the project will also be discussed.
- **Chapter 4: Results and Evaluation** — This chapter will document the experimental results for each of the classifications performed using the different machine learning models and compare the performance of BERT, TF-IDF and the LLM-based approaches. In addition, it will describe the relative merits and limitations of each approach.
- **Chapter 5: System Design and Implementation** — This chapter will provide an overview of the end-to-end system developed for real-time classification of social media posts. The best-performing model from the experiments will be implemented as a cloud-based inference service and will be integrated into a complete pipeline which will retrieve, preprocess and classify incoming social media posts.

- **Chapter 6: Conclusion and Future Work** — This chapter summarizes the contributions of this research, discusses limitations and practical implications, and outlines directions for future development.

2. LITERATURE REVIEW

The literature review portion of this chapter highlights past research and relates it to social media in disaster response, specifically to earthquakes, and natural language processing for disaster-related crisis informatics. Six categories are highlighted: the history of crisis informatics; models for explaining the role of social media in emergencies; crowd-sourcing and disaster-related mapping; near-real-time disaster assessments and situational awareness; disaster-related analytics and NLP; and current barriers to implementing social media for emergency responses.

2.1 History of Crisis Informatics

Crisis informatics emerged as a new area of study in the middle part of the 2000s when researchers recognised that information and communication technologies (ICTs) offered a tremendous opportunity to improve disaster response efforts. Palen and Liu [1] created the term "crisis informatics" and demonstrated how affected populations' use of the internet to communicate about disasters significantly changed the nature of emergency response. They hypothesised that individuals experiencing a disaster would move from being passive consumers of information provided by authorities to active participants in disaster response; this hypothesis was confirmed by subsequent disaster response efforts.

In a historical review of social media usage in emergencies from 2002–2017, Reuter and Kaufhold [2] identified several themes that consistently appeared in social media usage in emergency situations, and identified that social media usage in disaster response efforts have moved from experimental uses of SMS systems to multi-platform (Twitter and Facebook) usage in large-scale disaster responses. Reuter and Kaufhold's review of the past 15 years of the crisis informatics field illustrated the growth of the field, but also identified numerous gaps in crisis informatics research, particularly a lack of research on social media platforms used in languages other than English.

2.2 Conceptual Frameworks for Understanding Social Media Usage in Emergency Situations

A number of conceptual frameworks have been created to enable researchers to develop systematic analyses of the roles of social media in disaster response efforts. For example, Houston et al. [3] developed a functional framework that categorised social media usage in the disaster cycle (preparation, response, and recovery); the authors identified specific functions that social media can perform, including: dissemination and collection of disaster-related preparedness information; detecting and signalling disasters; requesting and receiving assistance; and documenting lessons learned from disasters. This framework has informed the development of subsequent research and has influenced best practices in emergency management.

Literature reviews have provided the theoretical basis for integrating social media into crisis communication. Veil et al. [4] evaluated the integration of social media into risk and crisis communication practices and identified the benefits and drawbacks for emergency management organisations. Simon et al. [5] provided an overview of how social media facilitates two-way communication between emergency management organisations and the public in emergency situations and thus differentiates social media from traditional one-way communication mediums.

At the policy level, Lindsay [6] evaluated the use of social media in emergency response in the United States at the time of her 2011 publication, and examined the implications of these emerging trends for emergency management policies. Lindsay's report provided some evidence for the value of platforms like Twitter and Facebook for situational awareness, but also raised questions about the reliability of information disseminated through social media and the possibility of exacerbating the digital divide. Alexander [7] expanded on this examination of the policy implications of the use of social media in emergency response by examining the ethical aspects of social media use in

emergency response, including privacy concerns, the management of data, and the dual ability of social media to either support or hinder affected communities.

Hughes and Palen [8] also investigated the role of emergency management organisations in using social media. They reported on the organisational changes associated with emergency management organisation Public Information Officer (PIO) use of social media, and found that PIOs showed passion for social media, but also expressed concern about the resources needed to utilise social media effectively and verify the accuracy of information disseminated through social media. These concerns were significant barriers to the adoption of social media by emergency management organisations.

2.3 Crowdsourcing and Crisis Mapping

Another development in crisis informatics is the advent of crowdsourced information collection and crisis mapping, often referred to as "crowdmapping." Okolloh [9] described the Ushahidi platform, which allowed citizens to submit crisis reports via the internet and SMS for aggregation and display on interactive maps. The Ushahidi platform was built in response to post-election violence in Kenya in 2008, and demonstrated the potential of citizen-led, volunteer-driven crisis mapping to augment official response efforts.

The 2010 Haiti earthquake served as a landmark in the use of crowdsourced disaster responses. Zook et al. [10] investigated how volunteers' provision of geographic information through Ushahidi and OpenStreetMap helped support the response effort, and how thousands of volunteers around the world made contributions to mapping efforts within days of the earthquake. Starbird and Palen [11] examined the formation of online networks by "voluntweeters," or digital volunteers who organised themselves on Twitter, in order to aggregate, validate and route information to responders, during the Haiti earthquake response.

Meier [12] documented the practice of digital humanitarianism, which includes crowdsourced crisis mapping and volunteer analysis of social media data. He views digital humanitarians as a new form of humanitarian response that complements official responders with networks of volunteers who can process and validate information at scale.

More recent work has examined the long-term feasibility and effectiveness of crowdmapping platforms. Ogie et al. [13] presented lessons from PetaJakarta, a three-year project to use Twitter to map flood incidents in Jakarta, Indonesia, and drew out lessons on public engagement, data credibility, and institutional factors affecting crowdmapping success.

2.4 Real-Time Event Detection and Situational Awareness

One of the most promising uses of social media in disaster contexts is real-time event detection and situational awareness. Earle et al. [14] discovered that spikes in Twitter activity preceded official seismic reports for some earthquakes, indicating that social media could potentially be a "social sensor" for quickly identifying an event. This discovery was significant because it implied that crowdsourcing from the population experiencing the disaster could provide information before official monitoring infrastructure did.

Sakaki et al. [15] extended this concept into a probabilistic model for detecting earthquakes in Japan based on Twitter data and sending notifications to users. The model was able to detect earthquakes sooner than many seismic networks for some events, and demonstrated the practical application of using social media for earthquake detection. This work demonstrated that Twitter can be a valuable addition to seismic monitoring in areas where there is little sensor coverage.

While detecting the occurrence of the event is important, social media may also be beneficial in creating situational awareness by providing ground-level information on how the impacts of the disaster are unfolding. Vieweg et al. [16] studied Twitter

messages during the 2009 Red River floods and Oklahoma wildfires, and determined that tweets contributing to crowdsourced disaster reporting often provided valuable information on damage to critical infrastructure, road closures, and evacuations that could enhance situational awareness for emergency responders. This work established the basis for extracting situational information from the chaotic stream of social media posts.

Using social media to rapidly assess damage from disasters, Fohringer et al. [17] assessed the use of geotagged social media photos and videos to map the extent of flooding rapidly, and demonstrated that crowdsourced imagery can be used to evaluate the magnitude of disaster impacts in near-real-time when official data is lacking. Kryvasheyeu et al. [18] demonstrated that the volume and spatial distribution of Twitter activity can serve as a proxy for assessing physical damage and power outages after disasters, and represent a new method for rapidly assessing damage using social media as big data.

2.5 Data Analytics and NLP for Disaster Response

Because the volumes of social media data generated during disasters continue to increase, machine learning and natural language processing techniques have become essential for extracting actionable information from social media data. Imran et al. [19] developed the AIDR platform that combines machine learning with human crowdsourcing to automatically categorise incoming social media messages during a crisis into useful categories (e.g., infrastructure damage, casualties and injuries, resource needs). AIDR was an important step towards operationalising social media analytics for emergency management.

Ashktorab et al. [20] developed Tweedr, a system that utilises natural language processing to extract actionable information from tweets, including resource needs and geographic information, to support crisis managers. Their work demonstrated the potential of NLP techniques to transform unstructured social media text into structured, actionable

intelligence for disaster response.

Comparative studies have also compared information patterns across different types of crises. Olteanu et al. [21] conducted a large-scale comparative study of Twitter data from 26 different crises, created a taxonomy of information types (e.g., alerts, damage reports, donations, sympathy, etc.), and analysed which types of information categories are common to natural versus human-induced disasters. This work provides valuable insight into the predictable patterns of crisis communication that can guide the design of classifiers.

More recently, Ray Chowdhury et al. [22] used a multilingual BERT-based model to classify disaster tweets into multiple overlapping categories where a single tweet could be reporting damage and requesting aid at the same time. In addition to their development of a method to improve model generalisability to new disasters and new languages, they provided a way for models to be operationally deployed.

Other survey articles summarise the current state of the art for machine learning for disaster management. Linardos et al. [23] provided an overview of methods and applications, including social media analytics, and described larger trends in the field such as deep learning and multimodal fusion. Linardos et al.'s [24] most recent article describes how large language models (LLMs) working in concert with traditional machine learning can be used to label and structure disaster-related social media data and how fine-tuning LLMs on disaster texts can assist in classifying situational information in tweets regardless of whether or not the tweet is written in a multilingual format.

2.6 Case Studies: Earthquakes and Regional Platforms

Social media has been studied in a number of case studies that describe its usage in specific earthquake events, and provide insight into platform-specific dynamics and differences across regions. Acar and Muraki [25] described the use of Twitter immediately

following the 2011 Tōhoku earthquake and tsunami in Japan and determined that Twitter was a very valuable medium for disseminating information and coordinating relief efforts but that rumours spread quickly through user communities. Peary et al. [26] examined the use of social media in both domestic and international contexts during the same event and showed that social media played a critical role in supporting a number of important activities including searching for the status of family members, identifying missing people, and organising volunteer efforts.

Ertuncay et al. [27] were one of the first to examine the usefulness of Ekşi Sözlük as a source of damage reports after earthquakes in Turkey and reported a high level of correlation between damage reports made by crowdsourcing through Ekşi Sözlük and official macroseismic intensity metrics. Their findings motivated the research in this thesis, which aims to develop natural language processing capabilities for processing disaster content in Turkish on Ekşi Sözlük.

2.7 Challenges and Limitations

Although social media has shown promise in assisting disaster management efforts, there are still important challenges to overcome before social media can be fully operationalised for disaster management purposes:

2.7.1 Misinformation and Rumour Propagation

Fast spread of misinformation during emergencies presents a serious problem. An early study of the spread of misinformation during the 2010 Chile earthquake was conducted by Mendoza et al. [28] who observed that true and false information spread differently and that social media communities frequently engaged in crowdsourcing correction of false information. Oh et al. [29] employed rumour theory to analyse Twitter data from the 2010 Haiti earthquake and identified why misinformation would spread and under what conditions the social media community would correct false information. These

studies illustrate the ongoing difficulty of establishing credibility of information in social media during emergencies.

2.7.2 Information Overload

A tremendous amount of social media postings occur during major emergencies, creating an overwhelming amount of data for both human analysts and automatic systems. Classifying and filtering millions of social media postings in near-real-time require sophisticated artificial intelligence (AI) systems, which in turn require well-labelled training data—a resource that is relatively rare for many languages and disaster types.

2.7.3 The Digital Divide

Van Dijk [30] illustrated long-standing disparities in access to the internet and digital literacy skills among various populations relevant to disaster management: not all disaster-affected populations are equally represented on social media, thus leaving behind vulnerable populations such as the elderly, rural residents, and low-income households. Social-media-based emergency response systems therefore face the possibility of systematically ignoring those who need the most assistance.

2.7.4 Platform Access and API Restrictions

As a result of growing concern about data privacy and misuses of social media data, social media platforms have increasingly restricted access to APIs for conducting research and operational uses of social media data. The restrictive API access policies imposed by Twitter (now X) since its 2023 acquisition have been particularly restrictive, resulting in scholars looking for other platforms with more liberal data access policies.

2.8 Research Gaps and Opportunities

The literature review identifies a number of gaps in the literature that support the need for the research being conducted:

- **Limited Turkish-language resources**: Although Turkey is considered to be a country with a significant seismic hazard and is home to a significant population of active social media users, there are limited amounts of Natural Language Processing (NLP) datasets available for Turkish disaster-related social media content. A review by Acıkara et al. [31] pointed to the dominant presence of English-language Twitter studies and called for more research on platforms and languages beyond Twitter.

- **Underutilisation of regional platforms**: While Ertuncay et al. [27] demonstrated the potential of Ekşi Sözlük for assessing earthquake intensity, no previous research appears to have developed NLP classifiers specifically designed for Ekşi Sözlük.

- **Limited multi-dimensional classification**: Many of the existing systems classify social media postings into a single category, while social media postings regarding emergencies often contain multiple forms of information (e.g., damage reports coupled with aid requests). The use of multi-label approaches like those employed by Ray Chowdhury et al. [22] has been relatively unexplored for Turkish-language content.

- **Disconnect between research and deployment**: While classification systems have been developed in research settings, few have been deployed as operational services that are able to process real-time social media streams.

In light of the gaps identified above, this thesis creates a large annotated Turkish disaster dataset from Ekşi Sözlük, explores multiple classification approaches (BERT, TF-IDF, and LLM-based), and deploys the best-performing models as a cloud-based inference

service designed for real-time classification.

3. METHODOLOGY

This chapter describes the methodological approach followed to achieve the research goals of this project. The methodology is explained as part of five Work Packages, WP1 through WP5, each containing one of the main components of the entire system. The Work Packages are: developing the platform; collecting and organizing said data; pre-processing said data; human-annotating said data; and training models to automatically assess earthquake damages through social media.

The layout is clear, moving from creating an infrastructure to collect data; to preparing the data; to annotating the data by humans; and finally to modeling social media data to perform an automatic earthquake damage assessment.

3.1 Work Package 1: Emergency Reporting Platform

An ad-hoc application would typically have been developed after an earthquake event to facilitate rapid communication among emergency personnel. The app developed as part of this research project has been constructed with the intention of being a purpose-built platform that considers accessibility and data privacy as part of the design process. The platform's source code will be published on GitHub to ensure transparency, and to allow researchers or emergency response agencies to modify the platform for their needs.

3.1.1 Platform Overview

The platform enables users who are affected by the earthquake to fill out an emergency report in a structured format. An emergency report includes the user's location, the user's observations of the damage to the building, and the user's immediate needs. A significant design decision was to utilize a single code base for all three target platforms: iOS, Android and Web Browsers. With the utilization of Expo and React Native, the development costs are lower, and the behavior of the platforms will be consistent across

devices.

In order to minimize friction and allow users to fill out and submit the report quickly during high stress situations, there is no registration or login requirement for users to complete and submit the report in less than one minute. The data collected and stored in a cloud database will be accessible for analysis and/or review by emergency personnel or analysts utilizing backend queries.

3.1.2 Technical Stack

The application is written entirely in TypeScript, utilizing the following components:

- **Frontend:** Expo (SDK 54) with React Native drives the client application. Expo Router manages screen navigation. The application will compile to native iOS and Android builds, as well as to a static web export using Expo.
- **Backend:** A Node.js serverless function deployed on Vercel receives form submissions over HTTPS POST requests. The function validates the incoming JSON payload, and adds CORS headers to enable access to the web client. The function generates a unique key for each submission using UUID v4, and then writes the record to the database. The backend code is minimal—approximately 50 lines—because the majority of the logic is contained within the client application.
- **Database:** All emergency reports are stored in a DynamoDB table named `EmergencyFormSubmissions`. The reasons for selecting DynamoDB include its high availability, ability to automatically scale, and low latency for write operations. The characteristics described above make it suitable for disaster response systems, which are subject to spikes in traffic from nearly zero to thousands of requests per minute following an earthquake. DynamoDB is able to handle these patterns without the need for the user to provision capacity manually.

- **Deployment:** Both the frontend (static web export) and backend (serverless function) are hosted on Vercel. The platform is integrated directly with the GitHub repository, and every time a commit is pushed to the main branch, Vercel automatically builds and deploys the code. The continuous deployment pipeline facilitates rapid development, bug fixing and the rollout of new features.

  A publicly accessible web based emergency reporting interface is deployed at:

  `https://earthquake-fe-psi.vercel.app/emergency`

This deployment represents a fully functional end-to-end pipeline, demonstrating that the architecture of the system is technologically viable and ready for production.

3.1.3 Data Collection

The emergency form, called "Acil Durum" (meaning "Emergency"), will collect the following fields:

- **User Information:** The user's name, surname, telephone number, and the Turkish National Identification Number (T.C. Kimlik No.) if provided. Telephone numbers represent the primary method for emergency responders to contact the individual.
- **User Location:** The user will enter as much or as little information about their location as they know including but not limited to, a full street address, a city name, or a neighborhood name.
- **Building Details:** The number of floors in the building, the type of building (i.e., concrete, adobe, or wood), and the structural status of the building (i.e., standing, damaged, or collapsed). Collecting building details allows emergency responders to prioritize areas with buildings that have collapsed or have been severely damaged.

- **Needs:** A dropdown list of the user's immediate needs such as food, water, heat, or shelter.
- **Hazards:** A dropdown list for reporting hazardous conditions such as gas leaks, fire, etc. Users will be able to select "none" if no hazardous conditions exist at the location.
- **Occupants:** The number of occupants at the location, which will assist emergency responders in estimating the amount of assistance that will be needed.
- **Medical Flags:** Four checkboxes for identifying individuals at the location who require urgent medical attention, are bleeding, are unconscious, or are in shock. The medical flags will enable medical personnel to triage incoming reports.

The emergency form is designed to strike a balance between collecting as much information as possible while still enabling users to quickly submit a report in the case of emergencies (Figure 3.1).

3.1.4 Architecture

The architecture of the system is a three-layer serverless architecture.

1. **Client Layer:** Users will interact with the application through either the mobile version (using Expo) or through a desktop web browser. The application will provide the emergency form to the user and perform client side validations (e.g., validate numeric fields contain only digits). When the user completes the form and clicks the "Submit" button, the client will create a JSON object that contains all of the field values and post the object to the backend endpoint via an HTTP POST request:

`https://earthquake-be.vercel.app/api/submit-form`

2. **Server Layer:** The Vercel serverless function will receive requests from the client and respond to OPTIONS preflight requests to enable CORS. For POST requests, the function will configure the AWS SDK using the credentials found in the environment variables and assign a UUID to the submission. The function will then call DynamoDB's `put` operation to save the submission as an item in the `EmergencyFormSubmissions` table. If the item is successfully saved, the function will return a 201 status code with a success message. Otherwise, the function will return a 500 status code with an error message.

3. **Database Layer:** DynamoDB will store each submission as a separate item in the `EmergencyFormSubmissions` table. Each item will contain the `submissionId` field (which is used as the partition key) and all of the fields submitted by the user. One of the standard functions of DynamoDB is to replicate data across multiple availability zones, thus ensuring durability and high availability without requiring any additional configuration.

Since this serverless architecture can scale dynamically, it is well-suited for use during disasters such as earthquakes. During normal times, the system will incur very little cost since no servers will be running. However, as soon as an earthquake occurs and traffic begins to increase dramatically, Vercel will automatically deploy additional function instances to accommodate the increased load and DynamoDB will scale its throughput to match. Once the traffic has decreased, the system will scale down.

3.1.5 User Interface

The design of the interface was guided by several principles:

- **Speed:** The fields were placed in a logical order so that users could quickly complete them without having to scroll back and forth (first the contact information, followed by the location, building details, needs, and medical flags).

- **Clarity:** All labels and placeholders are written in Turkish to better meet the needs of the primary language of the intended user base. Instead of picker wheels, the selection options for dropdown lists are presented as plain text to improve readability.
- **Accessibility:** The touch targets are sufficiently large to be easily tapped by users on a phone screen, even during stressful conditions or with shaky hands. The colors of the interface provide adequate contrast to facilitate readability.

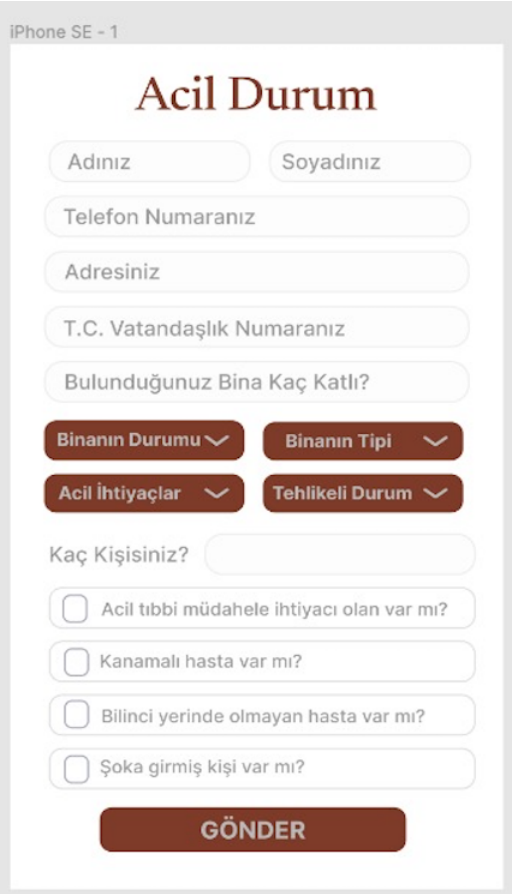

Figure 3.1: *The "Acil Durum" (Emergency) form in the mobile application.*

Once the user taps the "Gönder" (Send) button, the application will send the form data to the backend. After sending the form data, the application will display a success alert if the submission was successful, and will display an error alert if the submission

failed due to network connectivity issues or backend errors, and prompt the user to attempt the submission again.

3.1.6 Testing and Deployment

Testing and deployment occurred for this application on both iOS and Android systems using the simulator and also physical devices. Testing covered the following key areas:

- **Form validation:** Ensuring that numeric fields rejected non-numeric input and that the application would allow you to submit the form even if some fields were left blank.
- **Network Error Handling:** Ensuring that the application displayed suitable error messages either when the backend was inaccessible or returned an error message.
- **Multiple Submissions:** To verify that the backend and database could accept multiple submissions from users at the same time, a test was performed with multiple users submitting their forms at the same time and verified that there were no losses in data.
- **Platform Consistency:** Ensured that the layout and behavior of the application were the same on both iOS, Android and web browsers.

The web version of the application is available for public viewing. The mobile versions (iOS and Android) were tested internally; however, neither has been made available for download on the App Store or Google Play store. Availability to the app stores will be a future action once field testing of additional features and user feedback has been completed.

3.2 Work Package 2: Social Media Data Collection

This work package will focus on collecting posts related to earthquakes that have appeared on Ekşi Sözlük, a Turkish collaborative dictionary and social forum. This is different than what happened with Twitter, as after the 2023 policy changes, Twitter restricted access to the API and imposed expensive prices for accessing the API, while Ekşi Sözlük still allows for web scraping and is a good source for the amount of user generated content about disasters.

3.2.1 Data Source

Ekşi Sözlük organizes the content of users based on the concept of "başlık" (topic) pages. Users add "entry" posts to these pages to discuss the topic, and each entry includes the post content, the author's username, and the timestamp. In relation to earthquakes, users usually create topics with the title of the earthquake (for example, "6 şubat 2023 kahramanmaraş depremi") where users report their experiences during the earthquake, request assistance, describe damage to buildings and other structures, and comment on their feelings in regard to the event.

Because of its organization, Ekşi Sözlük is very effective for research about disasters: Posts appear in the order they were created, the timestamps for posts enable researchers to correlate them with the actual timeline of the event, and the başlık system enables researchers to group related posts without searching the entire site.

3.2.2 Implementation of Scraping

To automatically collect entries from the Ekşi Sözlük topics, a Python script was written. The script makes use of the following libraries and techniques:

- **BeautifulSoup:** Extracts entry data from the HTML content of each page. The script locates the `ul#entry-item-list` containing the entry list and then iterates

through each list item to extract the content, author username, and posting date.

- **Requests:** Used for making HTTP requests to get the page content. The script includes a custom User-Agent header to prevent the server from blocking the script.
- **Parallel Execution:** The `concurrent.futures.ThreadPoolExecutor` module enables the simultaneous (and therefore much faster) downloading of multiple pages. Therefore, the total time required for the scraping of a large topics with hundreds of pages is significantly reduced.

The script operates as follows:

1. The user enters the URL of the başlık as a command line parameter.
2. The script downloads the first page and determines the total number of pages from the pagination element so that the number of pages that need to be downloaded can be determined.
3. The script uses a thread pool to send requests to all pages concurrently (for example, `?p=1`, `?p=2`, ..., `?p=N`).
4. The script collects all entries for each page and saves each entry to a separate text file with the posting date and author username as part of the filename (for example, `06.02.2023_14:32_username.txt`).
5. All Twitter links found within the entries are collected and saved to a separate file for possible cross-platform comparisons.

Since each entry is saved as a separate file, downstream processing of the entries is greatly simplified since entries can be easily selected, sampled, or processed in parallel without loading the entire corpus into memory.

3.2.3 Entries Collected

The following topics related to earthquakes were scraped:

1. **1 Mayıs 2003 Bingöl Depremi** — 43 entries relating to the Mw (moment magnitude) 6.4 earthquake in Bingöl province, including the deaths of students in a collapsed dormitory.

2. **23 Ekim 2011 Van Depremi** — 931 entries relating to the Mw 7.1 earthquake in Van province, including personal accounts of damage, discussions of the government's response to the disaster, and descriptions of rescue operations.

3. **24 Ocak 2020 Elazığ Depremi** — 2,195 entries relating to the Mw 6.7 earthquake in Elazığ and Malatya provinces, including reports of the damage to buildings and infrastructure and descriptions of the rescue efforts.

4. **30 Ekim 2020 Ege Denizi Depremi** — 3,414 entries relating to the Mw 7.0 Aegean Sea earthquake that damaged buildings extensively in İzmir and triggered a tsunami.

5. **6 Şubat 2023 Deprem Yardım** — 10,701 entries focusing on coordinating aid, requesting rescues, and volunteering to assist those affected by the Kahramanmaraş earthquakes.

6. **6 Şubat 2023 Kahramanmaraş Depremi** — 7,616 entries covering a broader view of the earthquake sequence that occurred in February 2023, including reports of damage, personal accounts of surviving the earthquake, and comments from the general public.

Therefore, the total number of entries in the corpus is **24,900 entries** over a period of two decades representing various earthquakes in Turkey. The entries in the February 2023 topics make up approximately 74% of the total number of entries (18,317 entries).

A UUID was assigned to each entry to enable consistent referencing throughout the preprocessing, annotation, and model training phases.

3.2.4 Characteristics of the Data

The collected entries have many of the characteristics common to social media text:

- **Informal Language:** Users write in informal Turkish (colloquial), and often omit punctuation or use non-standard spellings.
- **ASCII Turkish:** Many posts use ASCII equivalents for Turkish characters (for example, “s” instead of “ş”, “i” instead of “ı”) because of older keyboard layouts and mobile input methods.
- **Mixed Content:** Entries vary from one sentence reactions to multi paragraph narratives. Some entries include only emotional expressions (for example, “Allah yardımımız olsun”), while other entries include detailed location information and descriptions of damage to buildings and other structures.
- **Time Clustering:** Most entries for each earthquake were entered within the first 48–72 hours, and most of the remaining entries were entered in subsequent days.

Therefore, a preprocessing pipeline is necessary (explained in Work Package 3) to normalize the text prior to training models with the text.

3.2.5 Ethical Considerations

Data collection was conducted with attention to ethical guidelines. All scraped content is publicly accessible on Ekşi Sözlük without requiring user authentication. No personally identifiable information beyond publicly visible usernames was collected. The data is used solely for academic research purposes, and no attempts were made to identify

or contact individual users. The scraping was performed at a moderate request rate to avoid overloading the platform's servers.

3.3 Work Package 3: Data Preprocessing and Corpus Development

This work package outlines the preprocessing pipeline developed to normalize raw social media text into a form suitable for machine learning. The pipeline is implemented in TypeScript and relies heavily on the NLPToolkit suite of libraries for Turkish language processing.

3.3.1 Pipeline Implementation Overview

The preprocessing code is organized as a Node.js project with the following key dependencies:

- **nlptoolkit-corpus:** Provides the `TurkishSplitter` class for sentence segmentation.
- **nlptoolkit-morphologicalanalysis:** Provides `FsmMorphologicalAnalyzer` for morphological analysis of Turkish words.
- **nlptoolkit-spellchecker:** Provides the `SimpleSpellChecker` for correcting misspelled Turkish words.
- **nlptoolkit-deasciifier:** Converts ASCII-only Turkish text to proper Turkish characters.
- **word-list:** An English dictionary used to identify English loanwords and code-switched terms.

Data passes through the pipeline in stages, with scripts generating intermediate files to be used as input to the next stage. This modular structure makes it possible to re-run individual stages without having to repeat the entire pipeline.

3.3.2 Corpus Creation and Text Preprocessing

The first script (`index.ts`) reads all text files from the six earthquake başlık folders and performs initial cleanup:

1. **Special character removal:** Punctuation marks, special characters, and symbols are removed using regular expressions. Characters such as parentheses, brackets, quotation marks, and mathematical symbols are removed entirely, while sentence-ending punctuation is replaced with spaces.

2. **URL removal:** URLs matching patterns like `www.*.com` are removed to eliminate embedded link noise.

3. **Number removal:** Numeric digits are removed, as they do not contribute to semantic analysis.

4. **Stopword filtering:** Platform-specific tokens (e.g., "edit") are removed.

After cleanup, the cleaned text from all files is passed through the `Turkish Splitter` class for segmentation into individual sentences. Each sentence is written to a combined output file, and all distinct words are collected into a separate vocabulary file for subsequent analysis.

3.3.3 Morphological Analysis

The second script (`morphological.ts`) performs morphological analysis on the distinct word list using `FsmMorphologicalAnalyzer`. For each word, the analyzer attempts to identify:

- The word's root/stem form

- Part-of-speech (POS) tag

- Morphological features such as tense, case, and number

Tokens that the analyzer cannot parse are written to a separate "unanalyzed tokens" file. After the initial pass, approximately 35% of the vocabulary remained unanalyzed (43,521 distinct tokens).

3.3.4 Iterative Refinement of Unanalyzed Tokens

Unanalyzed tokens were processed through multiple refinement stages, each implemented as a separate script:

1. **Capitalization correction (`capitalMorphological.ts`):** Many words failed analysis because Turkish morphological rules are case-sensitive. Each unanalyzed token was converted to title case (first letter uppercase) and re-analyzed. This step resolved a portion of proper nouns and sentence-initial words.

2. **English dictionary check (`englishCheck.ts`):** Remaining unanalyzed tokens were checked against an English dictionary (`word-list` package). Tokens identified as valid English words—common in Turkish social media due to code-switching—were marked as analyzed and excluded from further processing.

3. **Deasciification (`deasciifiedFix.js`):** Tokens written in ASCII-only Turkish (e.g., "gorus" instead of "görüş", "yardimlasma" instead of "yardımlaşma") were converted to proper Turkish orthography using `nlptoolkit-deasciifier`. The corrected forms were then re-analyzed morphologically.

4. **Spell checking (`spellchecked.ts`):** The `SimpleSpellChecker` was applied to remaining tokens to correct spelling errors. For each misspelled word, the spell checker suggests corrections based on edit distance and morphological validity. Corrected words were stored in a JSON mapping file for later corpus reconstruction.

5. **Context extraction (`sentenceSplitter.js`):** For words that still could not be analyzed, the pipeline extracted their surrounding context (10 words before and after) from the original sentences. This context was saved to a JSON file to assist with manual review.

Each stage produces a new unanalyzed words file with a sequential suffix (e.g., `unanalyzed_words2.txt`, `unanalyzed_words3.txt`), allowing progress to be tracked and individual stages to be debugged independently.

3.3.5 Results of Preprocessing

Following completion of all automated refinement stages, the number of unanalyzed tokens was reduced from 43,521 to 14,687—a 66% reduction. The remaining unanalyzed tokens consisted primarily of:

- Informal slang and neologisms specific to Ekşi Sözlük
- Intentional misspellings used for emphasis or humor
- Foreign words other than English (e.g., Arabic, Kurdish)
- Usernames and proper nouns not in the morphological lexicon

The remaining tokens were manually reviewed in batches. Where possible, corrections were added to a custom dictionary; otherwise, tokens were flagged for exclusion from training data.

3.3.6 Corpus Rebuilding

The final script (`replaceFromCorpus.js`) applies all corrections back to the original corpus. For each sentence in the combined output file, tokens are replaced with their corrected forms using the mappings from the spell-checking and deasciification

processes. The result is a cleaned corpus where the majority of words can be successfully analyzed morphologically.

3.3.7 Challenges

The preprocessing pipeline addressed many of the challenges associated with processing user-generated Turkish text:

- **Slang and colloquial language:** Ekşi Sözlük users employ informal expressions, abbreviations, and platform-specific jargon that do not appear in standard dictionaries.
- **Spelling variation:** The same word may be spelled differently across posts due to typographical errors or stylistic choices.
- **Code-switching:** Users frequently mix Turkish and English within a single sentence, requiring detection and separate handling of English tokens.
- **ASCII Turkish:** Many posts lack proper Turkish diacritics, making morphological analysis impossible without deasciification.

Because the pipeline was designed to be modular and iterative, each challenge was addressed systematically while maintaining the ability to audit and refine individual stages.

3.4 Work Package 4: Annotation of Social Media Data

This work package involved the manual annotation of the pre-processed corpus to develop labeled datasets for supervised learning. Two rounds of annotation occurred: the first round of primary labeling to categorize all entries based on content type; and the second round of secondary labeling to identify how severely damaged structures were described in the entries that contained descriptions of physical damage.

3.4.1 Scope and Constraints of Annotation

The entire corpus created by Work Package 2 has 24,900 entries. Time and financial constraints prevented the full corpus from being annotated, and therefore 18,628 entries—approximately 75% of the overall corpus—were used for annotation purposes. The annotated subset included only entries from the February 2023 Kahramanmaraş earthquakes because those were the most recent and operationally significant.

Two Microsoft Excel files are created for the annotation data:

- `Deprem Tam Data.xlsx`: Each of the 18,628 entries in this file were given primary labels including unique identifiers (UUID); original file name; entry text; and the assigned labels.

- `Deprem Hasarlı Data.xlsx`: Included all the 3,580 damage-related entries from the Deprem Tam Data.xlsx file that were subsequently selected for secondary labeling for damage severity (AOD) classification.

3.4.2 Annotation Process and Quality

The annotation was carried out by the author, who is a native Turkish speaker with familiarity in earthquake-related discourse from social media. Ideally, multiple annotators would have been employed to calculate inter-annotator agreement metrics such as Cohen's kappa; however, resource and time constraints made this infeasible. To reduce the risk of subjective bias, a set of annotation guidelines was prepared before labeling commenced, including explicit definitions for each category and illustrative examples. In cases where the content of an entry was ambiguous or did not clearly fit a single category, the more conservative label was assigned—N (None) for unclear content type and AH (Mildly Damaged) for uncertain severity—to avoid over-reporting damage or assistance needs.

3.4.3 Stage 1: Primary Labeling

Entries in the pre-processed corpus were categorized during primary annotation according to their content-type and whether it was relevant to earthquake response. The primary labeling system employs a single letter coding that can be combined to signify that an entry has multiple content-types:

- **N (None):** Entries that either do not pertain to the event or do not include actionable information. This category includes generalized commentary; jokes; political opinions; and off-topic discussion.
- **H (Hasarlı / Damage):** Entries that describe physical damage to buildings, infrastructure or the environment. Examples include reports of collapsed structures, cracked walls or damaged roads.
- **Y (Yardım / Assistance):** Entries containing direct requests for assistance or descriptions of urgent requirements. Often, these posts will include geographic coordinates and requests for rescue, medical care, food, etc.
- **B (Bilgi / Information):** Entries that include generalized or factual information about the earthquake, i.e.: magnitude; affected area(s); official announcements; coordination efforts.

Since entries may have multiple content-types, entries are labeled using combinations:

- **HY:** Both damage description and assistance request
- **HB:** Both damage description and informational content
- **YB:** Both assistance request and informational content

- **HYB:** All three content-types are present

See Table 3.1 for the breakdown of the primary label distribution of the annotated dataset.

Table 3.1: *Distribution of primary annotations in the dataset.*

Label	Description	Count	%
N	None / Irrelevant	10,405	55.9%
H	Damage	2,908	15.6%
Y	Assistance	2,364	12.7%
B	Information	2,204	11.8%
HY	Damage + Assistance	668	3.6%
YB	Assistance + Information	74	0.4%
HB	Damage + Information	3	0.02%
HYB	Damage + Assistance + Information	2	0.01%
Total		**18,628**	**100%**

The results indicate that greater than half of the entries (55.9%) were labeled as "none" (N). This is reflective of the nature of social media communication, where most of the posts consist of emotional responses and/or generalized commentary and relatively few contain actionable information. Of the 44.1% of entries that did contain at least one relevant content-type, damage reports (H) represent the most common single content-type.

3.4.4 Stage 2: Damage Severity Classification

All entries labeled with the value "H" (Damage)—regardless of whether the entry had a combination label of HY, HB, or HYB—were identified and labeled for secondary labeling of damage severity. A total of 3,580 entries were identified for secondary classification based on severity of damage.

Initially, the annotation framework provided a six-level severity rating. However, upon completion of the primary annotation, it became apparent that only four of the six categories (Y, OH, B, N) had fewer than 15 entries combined. Therefore, the original annotation framework was modified to include only two operationally feasible categories:

- **AH (Az Hasarlı / Mildly Damaged):** Minimal or limited damage that does not imply structural failure. Examples include cracked windows; fallen plaster; minor wall cracks; or reports that buildings continue to stand after the earthquake although they have shaken.

- **ÇH (Çok Hasarlı / Severely Damaged):** Significant structural damage that poses safety risks. Includes collapsed buildings; pancake collapse; buildings leaning precariously; or explicit report of individuals trapped under debris.

See Table 3.2 for the damage severity label distribution.

Table 3.2: *Distribution of damage severity annotations.*

Label	Description	Count	%
AH	Mildly Damaged (Az Hasarlı)	540	15.1%
ÇH	Severely Damaged (Çok Hasarlı)	3,008	84.0%
Other	Excluded categories (Y, OH, B, N)	32	0.9%
Total		**3,580**	**100%**

The results show a nearly 85/15 split towards ÇH (Severely Damaged). The results reflect the impact of the February 2023 earthquakes: the Mw 7.8 and Mw 7.5 events caused devastating damage to 11 provinces, and users were far more likely to report severe destruction than minimal damage. The extreme imbalance in the classes is addressed during the model training phase through stratified sampling and class weighting.

3.4.5 Characteristics of the Dataset

In addition to categorizing entries by type, several other trends were observed during the annotation process that may be important to consider when developing future downstream models:

- **Prevalence of Multiple Labels:** Less than 4% of entries were required to use

combination labels (HY, HB, YB, HYB). It appears that most posts focus on only a single content-type.

- **Geographic Location References:** Damage reports (H) and assistance requests (Y) frequently included geographic location information—street names, building names, or neighborhood designators—that could be valuable for named-entity recognition in future research.

- **Temporal References:** Many entries referenced relative time (i.e., "şu an", "biraz önce") rather than absolute timestamp references, making temporal analysis difficult.

- **Indicators of Ambiguity:** Posts often included hedge language (i.e., "söylentilere göre", "duyduğuma göre"), that indicated the source of unverified information, and may be useful for assessing credibility in future research.

3.4.6 Binary Classification Formulation

To facilitate model training, the multi-label annotation format was reduced to three separate binary classification problems:

1. **H vs. NH (Damage):** If an entry includes an "H" component in its label (H, HY, HB, HYB), it is labeled as positive; otherwise, the entry is labeled as negative.

2. **Y vs. NY (Assistance):** If an entry includes a "Y" component in its label (Y, HY, YB, HYB), it is labeled as positive; otherwise, the entry is labeled as negative.

3. **B vs. NB (Information):** If an entry includes a "B" component in its label (B, HB, YB, HYB), it is labeled as positive; otherwise, the entry is labeled as negative.

These three formulations simplify the classification problem and allow for identifying posts that contain multiple content-types by creating three independent classification outputs.

3.5 Work Package 5: Model Development and Evaluation

This work package includes the development, evaluation, and deployment of machine learning models for two classification problems: 1) sentence classification to identify damage reports, requests for assistance, and informational content; and 2) damage severity classification to distinguish between mildly damaged and severely damaged structures. Two modeling approaches were compared: traditional TF-IDF baseline with linear classifier and fine-tuning of Turkish BERT models for sentence classification.

All experiments were completed in Google Colab notebooks:

- `UpdatedTamDepremTfidf.ipynb`: TF-IDF baseline for sentence classification
- `UpdatedTamDepremBert.ipynb`: BERT models for sentence classification
- `UpdateHasarlıDepremTfidf.ipynb`: TF-IDF baseline for damage severity
- `UpdatedHasarlıDepremBert.ipynb`: BERT model for damage severity

3.5.1 Sentence Classification Models

The sentence classification task was defined as three independent binary classification problems. Each of the binary classification problems addresses a different operational question:

1. **H vs. NH (Damage):** Are there mentions of physical damage to buildings or infrastructure in the sentence?
2. **Y vs. NY (Assistance):** Does the sentence indicate a request or offer for assistance?
3. **B vs. NB (Information):** Does the sentence provide factual or contextual information about the earthquake?

Binary labels were created from the original multi-label annotations: if an entry contains "H" in its label (H, HY, HB, HYB), then the entry is positive for the damage classification task; similarly for "Y" and "B". The label "N" (None) was mapped to negative for all three classification tasks.

3.5.1.1 Dataset Preparation

Annotated data (`Deprem Tam Data.xlsx`) was uploaded to the Colab environment. The original `Label` column was transformed to create three additional binary target columns (`label_H`, `label_Y`, `label_B`). Missing values in text or labels were removed from all rows, and text fields were converted to strings.

The resulting distributions of the classes are heavily weighted towards the negative labels:

- H vs. NH: 3,581 positive (19.2%) vs. 15,047 negative (80.8%)
- Y vs. NY: 3,108 positive (16.7%) vs. 15,520 negative (83.3%)
- B vs. NB: 2,283 positive (12.3%) vs. 16,345 negative (87.7%)

The datasets for each task were divided using a stratified 90/10 train-validation split so that class ratios would remain consistent across both splits.

3.5.1.2 TF-IDF Baseline

As a reference point, a TF-IDF vectorizer was defined with unigrams and bigrams (`ngram_range=(1,2)`) and a maximum vocabulary of 50,000 features. A Linear Support Vector Machine (LinearSVC) with class-weight balancing was trained using TF-IDF vectors.

3.5.1.3 BERT Models

The `dbmdz/bert-base-turkish-cased` model was employed as the base encoder for the transformer approach. The model has been pre-trained on a large Turkish corpus and therefore able to process Turkish morphology, casing and informal vocabulary.

Tokenization Parameters The tokenization parameters were configured as follows:

- Maximum sequence length: 128 tokens
- Padding: to maximum length
- Truncation: enabled

Three separate `AutoModelForSequenceClassification` models were instantiated, one per binary task, each with `num_labels=2`. The pre-trained weights were loaded and only the classification head was randomly initialized.

Training Hyperparameters The training hyperparameters were defined similarly for all three tasks:

- Learning rate: 2×10^{-5}
- Batch size: 16 (training), 32 (evaluation)
- Epochs: 3
- Weight decay: 0.01

The training loop was handled by the Hugging Face `Trainer` API. Accuracy and macro-averaged F1-score were used as evaluation metrics; macro-averaging assigns equal weight to contributions of both classes irrespective of class ratios.

The trained BERT models were exported to directories (`tamDepremBert_H`, `tamDepremBert_Y`, `tamDepremBert_B`) and saved to Google Drive for deployment. Results of experimental comparisons among BERT, TF-IDF, and LLM baselines are reported in Chapter 4.

3.5.2 Damage Severity Classification Model

A second classification task was implemented to assess the severity of damage referred to in posts previously identified as damage-related. The task makes use of the secondary annotations from `Deprem Hasarlı Data.xlsx`, filtered to include only the two most prevalent classes:

- **AH (Az Hasarlı / Mildly Damaged):** Structural issues (cracks, fallen plaster, etc.) but no serious structural damage.
- **ÇH (Çok Hasarlı / Severely Damaged):** Serious structural damage; collapse risk or confirmed collapses.

After filtering, 3,547 entries remained: 539 AH (15.2%) and 3,008 ÇH (84.8%). The extreme class skewness is due to the catastrophic nature of the February 2023 earthquakes.

3.5.2.1 TF-IDF Baseline

The same TF-IDF settings were applied: unigrams and bigrams, 50,000 features, and a Logistic Regression classifier with class-weight balancing.

3.5.2.2 BERT Model

The `dbmdz/bert-base-turkish-cased` encoder was fine-tuned with a two-class classification head. The same hyperparameters used for the sentence classification models were applied for training. Label mappings (`label2id`, `id2label`) were

stored along with the model for future inference.

The trained model was saved to `bert-hasarli-final/` and exported as a zip file for deployment. The experimental results for the damage severity classification task are presented in Chapter 4.

3.5.3 Prompt-based Zero-Shot LLM Classifier

In addition to the supervised transformer models, another baseline utilizing a large language model (LLM) was created as a zero-shot model based upon a custom prompt to enable a structured approach for asking all four classification questions at once. The objective of this prompt-based approach was to determine whether a single well-designed prompt could elicit all four classifications necessary for this thesis:

- **H vs. NH:** Does the sentence refer to physical damage (**H**) or not (**NH**)?
- **Y vs. NY:** Does the sentence refer to a request or offer for assistance (**Y**) or not (**NY**)?
- **B vs. NB:** Does the sentence represent factual or contextual information (**B**) or not (**NB**)?
- **AH vs. ÇH:** If the sentence refers to damage, is the damage minor (**AH**) or major (**ÇH**)?

The prompt-based setup does not need to fine-tune the LLM; rather, it asks the LLM to classify the input sentences with a single prompt consisting of three main parts:

1. Definitions of each label;
2. Explicit statement of what the LLM should return as output;
3. JSON response schema.

For each input sentence, the LLM receives the text and returns a JSON object representing the LLM's prediction for all four classifications: H or NH, Y or NY, B or NB, and AH or ÇH.

The core prompt used in the experiments is illustrated below.

```
You are an assistant that classifies Turkish social media posts
 ↪  regarding earthquakes.

For each given sentence, you must decide:

1) Whether it describes PHYSICAL DAMAGE:
   - H  = the sentence describes any physical damage to
    ↪  buildings or infrastructure.
   - NH = the sentence does NOT describe physical damage.

2) Whether it contains an ASSISTANCE REQUEST or OFFER:
   - Y  = the sentence contains a request or offer for help,
    ↪  resources, or rescue.
   - NY = the sentence does NOT contain such a request or offer.

3) Whether it provides INFORMATION:
   - B  = the sentence provides factual or contextual
    ↪  information
          (e.g., magnitude, location, time, situational
           ↪  updates).
   - NB = the sentence does NOT provide such information.
```

```
4) If the sentence describes damage (H), estimate DAMAGE
↪  SEVERITY:
   - AH = Az Hasarli (mildly damaged): minor cracks,
    ↪  non-critical damage, cosmetic issues.
   - CH = Cok Hasarli (severely damaged): serious damage, risk
    ↪  of collapse, or collapsed buildings.

Important rules:
- Every sentence must be labeled as H or NH, Y or NY, and B or
↪  NB.
- If the sentence is NH (no damage), you should still output a
↪  severity label,
  but set it to "AH" if it implies mild impact, or "CH" if it
   ↪  clearly indicates severe impact.
  If there is truly no damage, choose "AH" by default as the
   ↪  neutral / non-severe option.

Return your answer ONLY as a JSON object with the following
↪  fields:

{
  "damage": "H or NH",
  "assistance": "Y or NY",
  "information": "B or NB",
  "severity": "AH or CH"
}
```

```
Do NOT add explanations or any text outside the JSON.

Next, classify the following sentence:

[SENTENCE]
```

When processing each Ekşi Sözlük sentence, the placeholder `[SENTENCE]` is replaced with the actual Ekşi Sözlük sentence to be classified. The model's answer is then parsed as a JSON and the four fields are linked to the four binary targets that have been used throughout the entire project:

- `"damage"` → H vs. NH,
- `"assistance"` → Y vs. NY,
- `"information"` → B vs. NB,
- `"severity"` → AH vs. CH.

3.5.3.1 *

Implementation Details

The LLM classification pipeline is implemented as a Node.js script (`hasar-Tespit/index.js`) for interacting automatically with the OpenAI Chat Completions API. The implementation utilizes the following libraries:

- `axios`: HTTP client library for handling API calls
- `uuid`: Used to generate unique IDs for each classified item
- `csv-writer` and `csv-parser`: For managing CSV input and output for storing results and continuing where left off

The pipeline is run as follows:

1. **File detection.**

   The script recursively searches the preprocessed corpus directory for files with names matching the pattern `*_corrected.txt`. These are the cleaned and normalized text files produced by the preprocessing pipeline in Work Package 3. Each file represents a single entry and is processed as a single classification unit.

2. **Request preparation for API call.**

   For each file, the script creates a chat-completion request with two messages:

   - A `system` message with the classification prompt including label definitions and output format specifications.
   - A `user` message prefaced with `"TEXT: "` followed by the content of the entry.

   The `gpt-4o-mini` model was selected due to budget constraints and is employed with `temperature=0` to produce determinate and reproducible answers. The `max_tokens` parameter is set to 200 which is enough space to hold the JSON response. It should be noted that using a larger model such as GPT-4, or employing few-shot prompting with domain-specific examples, could potentially yield improved zero-shot performance; however, this was not feasible within the scope of this project and represents a limitation of the current LLM comparison.

3. **Parsing and validation of response.**

   Although the prompt explicitly specifies returning a plain JSON object, the model sometimes produces responses that are wrapped in Markdown code fences (e.g., ```` ```json ... ``` ````) or have other minor formatting errors. The script resolves the different cases using a multi-step parsing process:

- Remove Markdown code fences and surrounding whitespace.
- Extract the substring bounded by the first { and last }.
- Try to parse the JSON; if unsuccessful, perform regular-expression repairs to quote unquoted label values (e.g., `"damage": H` → `"damage": "H"`).
- Validate that the four required fields (`damage`, `assistance`, `information`, `severity`) are present and contain string values. If validation fails, the entry is logged as an error.

4. **Resumable batch processing.**

   To avoid re-classifying sentences that have already been processed in an earlier run, the Node.js program stores previous predictions in a CSV file:

   `llm_sentence_and_severity_results.csv`

   When a new run begins, the CSV file is read and the set of previously processed filenames is loaded into memory. Any file with a relative path appearing in this set is skipped. This design makes the pipeline resumable: if the program is interrupted (due to rate limits, etc.), it can resume processing at the point where it was interrupted.

5. **Error and rate limit handling.**

   Each API call is wrapped in a `try--catch` block. If an exception occurs, the program logs a diagnostic message including the filename and error code. In particular, if the API returns a `rate_limit_exceeded` error, the program stops processing gracefully so that it can be restarted later without losing any already obtained results.

6. **Storage of prediction results.**

   For each successfully classified entry, the script creates a record with the following fields:

- `id`: A unique identifier (UUID v4) for the classification
- `filename`: The relative path to the source file
- `original_text`: The raw text content of the entry
- `damage`: Predicted label (H or NH)
- `assistance`: Predicted label (Y or NY)
- `information`: Predicted label (B or NB)
- `severity`: Predicted label (AH or ÇH)

Records are appended to the output CSV file, `llm_sentence_and_severity_results.csv`, using the `csv-writer` library. The script can process up to 30,000 files per run, as defined by the `MAX_FILES` constant.

This implementation transforms the prompt-based LLM into a practical zero-shot baseline that can be applied to thousands of sentences with minimal manual intervention. It also illustrates common engineering challenges when deploying LLMs for large-scale classification: enforcing a strict output schema, resolving occasional formatting issues, managing API rate limits, and developing resumable, fault-tolerant batch-processing pipelines.

4. RESULTS AND EVALUATION

4.1 Experimental Results and Evaluation

The findings in this section present an empirical evaluation of how the three baseline methods perform for the task of sentence classification: BERT, TF-IDF+Linear SVM, and Zero-Shot LLM Classifier. The performance of all three models was assessed on the same stratified validation set consisting of 1,863 sentences.

4.1.1 Performance of BERT Model

For each of the three tasks—determining whether a sentence contains damage-related content (H vs. NH), assistance-related content (Y vs. NY), or informational content (B vs. NB)—three independent binary BERT classifiers were trained and evaluated. Overall performance metrics are reported in Table 4.1.

Table 4.1: *Overall performance of the three BERT binary classification models.*

Task	Accuracy	Macro F1-score
H vs. NH	0.9752	0.9599
Y vs. NY	0.9515	0.9128
B vs. NB	0.9520	0.8855

Detailed per-class breakdowns for each task are presented in Tables 4.2, 4.3, and 4.4. The data from these tables demonstrate that the model performs well and consistently across each of the three tasks. BERT's performance is particularly robust when distinguishing between physical damage (H) and non-damage (NH) content.

4.1.2 TF-IDF Baseline Performance

The performance results for the TF-IDF baselines are reported using unigrams and bigrams as features and Linear SVM as the classifier. The summary of overall performance for the TF-IDF baselines can be found in Table 4.5.

Table 4.2: *Per-label performance for the BERT H vs. NH classifier.*

Label	Precision	Recall	F1-score	Support
NH	0.9827	0.9866	0.9847	1496
H	0.9432	0.9274	0.9352	358
Accuracy			0.9752	
Macro F1			0.9599	

Table 4.3: *Per-label performance for the BERT Y vs. NY classifier.*

Label	Precision	Recall	F1-score	Support
NY	0.9702	0.9715	0.9709	1543
Y	0.8576	0.8521	0.8548	311
Accuracy			0.9515	
Macro F1			0.9128	

Table 4.4: *Per-label performance for the BERT B vs. NB classifier.*

Label	Precision	Recall	F1-score	Support
NB	0.9683	0.9772	0.9728	1626
B	0.8263	0.7719	0.7982	228
Accuracy			0.9520	
Macro F1			0.8855	

Table 4.5: *Overall performance of the three TF-IDF + Linear SVM models.*

Task	Accuracy	Macro F1-score
H vs. NH	0.9699	0.9521
Y vs. NY	0.9313	0.8768
B vs. NB	0.9308	0.8449

Results for the TF-IDF baselines broken down per label can be seen in Tables 4.6, 4.7, and 4.8. Overall, the TF-IDF baselines performed reasonably well; however, they performed less well than the BERT baselines, particularly on the minority positive classes.

Table 4.6: *Per-label performance for the TF-IDF H vs. NH classifier.*

Label	Precision	Recall	F1-score	Support
NH	0.9846	0.9781	0.9813	1505
H	0.9103	0.9358	0.9229	358
Accuracy			0.9699	
Macro F1			0.9521	

Table 4.7: *Per-label performance for the TF-IDF Y vs. NY classifier.*

Label	Precision	Recall	F1-score	Support
NY	0.9594	0.9581	0.9587	1552
Y	0.7923	0.7974	0.7949	311
Accuracy			0.9313	
Macro F1			0.8768	

Table 4.8: *Per-label performance for the TF-IDF B vs. NB classifier.*

Label	Precision	Recall	F1-score	Support
NB	0.9665	0.9541	0.9603	1635
B	0.6988	0.7632	0.7296	228
Accuracy			0.9308	
Macro F1			0.8449	

4.1.3 Performance of the LLM Baseline

A zero-shot Large Language Model (LLM) baseline was tested by evaluating its ability to classify text into the appropriate categories of assistance (**Y**), information (**B**), and level of damage (**H**). A prompt was developed that would enable the model to simultaneously classify text into all three categories. Once the model had classified the text, the output was converted to JSON format and compared against the labeled data.

Three independent binary classification tasks were developed based on the categories used in the Deprem Tam Data. These included: *damage (H)* versus *no damage (NH)*, *assistance (Y)* versus *no assistance (NY)*, and *information (B)* versus *no information (NB)*. Precision, recall, and F1-scores were calculated for each task using the labeled data from

the Deprem Tam Data.

Damage Classification (H vs. Not H) The model achieved an accuracy of 0.798 for damage classification (H vs. Not H). The model accurately predicted the negative class (Not H) with good precision (0.918) and recall (0.824). However, when attempting to predict the positive class (H), the model performed significantly poorer, with precision of 0.483 and recall of 0.690.

Table 4.9: *Binary metrics for damage classification (H vs. not H).*

Class	Precision	Recall	F1-score	Support
0 (not H)	0.918	0.824	0.869	15037
1 (H)	0.483	0.690	0.568	3578
Accuracy				0.798
Macro avg	0.700	0.757	0.718	18615
Weighted avg	0.834	0.798	0.811	18615

Assistance Classification (Y vs. Not Y) The class imbalance in the Assistance dimension is evident in the overall model accuracy of 0.501. The model frequently predicts the positive class (Y), achieving an extremely high recall (0.996); however, it achieves only a very low precision (0.250), in addition to a sharp decline in negative-class recall to 0.402.

Table 4.10: *Binary metrics for assistance classification (Y vs. not Y).*

Class	Precision	Recall	F1-score	Support
0 (not Y)	0.998	0.402	0.573	15508
1 (Y)	0.250	0.996	0.400	3107
Accuracy				0.501
Macro avg	0.624	0.699	0.487	18615
Weighted avg	0.873	0.501	0.545	18615

Information Classification (B vs. Not B) The performance in the information dimension was by far the worst of all three binary classification tasks. Although the precision for the negative class (0.885) is relatively good, it achieves only a 0.409 recall rate. Conversely, the positive class (B) has a very poor precision value (0.127).

Table 4.11: *Binary metrics for information classification (B vs. not B).*

Class	Precision	Recall	F1-score	Support
0 (not B)	0.885	0.409	0.559	16335
1 (B)	0.127	0.618	0.211	2280
Accuracy				0.435
Macro avg	0.506	0.513	0.385	18615
Weighted avg	0.792	0.435	0.517	18615

Although the LLM demonstrates good contextual comprehension, it struggles with composite labels such as HY, HB, and YB due to their infrequent occurrence and ambiguous nature in language. Although the LLM provides an informative upper bound of what is feasible using only prompts in terms of classification performance, its performance

still trails behind that of supervised BERT models.

4.1.4 Results Discussion

The evaluation of the performance of the three different sentence classification approaches—BERT, TF-IDF + Linear SVM, and a zero-shot Large Language Model (LLM)—provides a comprehensive overview of the advantages and disadvantages of the different methods used to classify earthquake-related social media content. This section synthesizes the evaluation of the performance of all three models and assesses their real-world applicability for rapid disaster response.

4.1.4.1 *

Comparing Overall Performance

BERT achieved the highest levels of accuracy and macro-F1 scores for each of the three binary tasks (H vs. NH, Y vs. NY, and B vs. NB) compared to all other tested methods. The strong performance of the BERT-based classifier demonstrates its ability to effectively understand the finer points of context in Turkish social media, including informal grammar, spelling variations, and domain-specific vocabulary. Although the TF-IDF-based models performed well in terms of accuracy, especially for the Damage Classification task, they performed poorly in terms of macro-F1 scores; thus, it can be inferred that these models had difficulty recognizing minority class patterns. The results also demonstrate that the LLM baseline model was significantly weaker than the BERT-based model in tasks requiring subtle distinctions or composite semantics, illustrating the limitations of using prompt-based classification to identify fine-grained operational categories.

4.1.4.2 *

Insights from Class-Specific Performance The H vs. NH task was the most successful among all the methods evaluated; the macro F1 scores for both BERT and TF-IDF exceeded 0.95. The data indicate that many terms referring to damage (e.g., *yıkıldı*, *çöktü*, *çatlak*, *bina hasarlı*) are distinct enough from terms present in general discussions that these two categories can be readily separated by any method.

On the contrary, the Y vs. NY and B vs. NB tasks were difficult for both BERT and TF-IDF. Requests for assistance are linguistically varied, and they can occur in two types of forms: explicit (e.g., "acil yardım lazım") or implicit (e.g., "kimse gelmedi daha"). The variety in how assistance requests are written made it difficult for both BERT and TF-IDF to identify the Y class reliably. Neighbourhood informational posts (B) also have a large semantic scope. The scope ranges from earthquake magnitudes to neighbourhood-level comments, making it difficult to determine what is an informational post and what is not based on sparse lexical features. In this case, BERT's contextual embeddings clearly outperformed TF-IDF.

The LLM baseline performed best on the categories that had common-sense semantics (i.e., obvious damage descriptions), but did poorly on fine-grained distinctions, such as determining if a comment was factual versus indirect commentary or determining if a request for assistance was a genuine plea versus an expression of emotion. These data illustrate the challenges of using zero-shot prompts for domain-specific classification, especially when the boundaries between categories are vague or specific to the dataset.

4.1.4.3 *

The Class Distribution Problem All of the above models show how the class distribution can affect the minority positive class performance (e.g., how many examples fall into each of the positive and negative categories). In addition, the two tasks (Y vs. NY

and B vs. NB) both have a much larger number of examples in the negative class than in the positive class. Although BERT does provide some degree of contextual understanding which can help alleviate the problems created by the class distribution issue, it continues to struggle with recall on the minority positive classes. The same was true for the TF-IDF model with a class weighted approach. Therefore, it appears that sparse lexical representations cannot adequately account for skewed distributions. The LLM also defaults to considering the majority class when it interprets content.

As such, these results emphasize the need for new methodologies to address the issue of class distribution in this domain such as through methods of data augmentation, oversampling, or multi-label modeling to capture the overlap between minority categories in the context of disaster-related discourse.

4.1.4.4 *

Operational Considerations From a practical standpoint, the best performing model to automate the triaging of user-generated posts during an active earthquake disaster response is BERT due to its superior performance in detecting content associated with physical damage (the category most relevant to emergency responders' decisions). Although TF-IDF's performance is unexpectedly high for its simplicity, it does not possess the level of robustness needed to support real-time deployment. The LLM's ability to reason semantically provides potential use as a secondary system for reviewing difficult classifications or supporting human-in-the-loop workflows.

4.1.4.5 *

Summary In summary, the results of the sentence classification experiments demonstrate that transformer-based models can effectively process disaster-related social media. BERT performed better than both TF-IDF and the zero-shot LLM on all three classification tasks, especially in identifying very subtle forms of assistance requests or informational

content. Overall, the results highlight the importance of representing the minority classes and provide evidence that a multi-label approach may be useful in capturing the multi-dimensional nature of communication during a disaster.

4.2 Performance of the Damage Severity Classification Model

This section will examine how well the three different models—BERT, TF-IDF, and a zero-shot LLM—are able to classify the severity of damage based on whether an example falls into one of the two categories: AH or ÇH.

4.2.1 BERT Model Performance

The BERT classifier shows strong performance on the binary severity classification task. Overall performance is summarized in Table 4.12.

Table 4.12: *Overall performance of the BERT damage severity model.*

Metric	Value
Accuracy	0.9042
Macro F1-score	0.8248
Validation Samples	355

A breakdown of per-class results from the BERT classifier are in Table 4.13.

Table 4.13: *Per-label precision, recall, and F1 for the BERT damage severity model.*

Label	Precision	Recall	F1-score	Support
AH (Mild)	0.66	0.76	0.71	54
ÇH (Severe)	0.96	0.93	0.94	301

4.2.2 TF-IDF Baseline Performance

A second classifier was evaluated using TF-IDF representations for the same dataset with a logistic regression model, in addition to the original Support Vector Machine classifier. The performance of this new baseline was significantly better than that of the

previous SVM classifier. In particular, the new baseline performed best when evaluating the minority *AH* (mild damage) class.

The results of the TF-IDF representations with logistic regression are presented in Table 4.15.

Table 4.14: *Per-class performance of the TF-IDF + Logistic Regression model.*

Label	Precision	Recall (Per-class Acc.)	F1 Score	Support
AH (Mild)	0.5806	0.6667	0.6207	54
ÇH (Severe)	0.9386	0.9136	0.9259	301

Table 4.15: *Traditional Baseline Performance (TF-IDF + Logistic Regression) for Damage Severity.*

Accuracy	0.8761
Macro F1 Score	0.7733

As compared with the previously discussed SVM traditional baseline (Accuracy = 0.8134; Macro F1 = 0.3606), the logistic regression model is a very competitive baseline in terms of both accuracy and macro F1 score. The increase in macro F1 score is large—over twice as much (0.7733 vs. 0.3606)—demonstrating a substantial increase in the ability of the model to be sensitive to the minority class. However, the overall performance of the logistic regression model continues to be less than that of the fine-tuned BERT classifier, especially when it comes to identifying the more subtle language features that indicate structures that were moderately damaged.

4.2.3 LLM Baseline Results—Severity Annotation Task

To assess the capability of the LLM to distinguish between two damage severity categories (*Ağır Hasarlı* (AH) and *Çok Hasarlı* (ÇH)), a severity annotation task was conducted. A total of 3,546 data points were available for this assessment, of which 540 belonged to the AH category and 3,006 belonged to the ÇH category, resulting in a

significant class imbalance. The results of the model's performance on this classification problem are summarised in Table 4.16. The overall accuracy of the model was 0.728, indicating that the model is capable of reasonably accurately assessing the general severity level of damage.

However, the model's performance varied significantly depending on the class. In particular, the model achieved very high precision for the ÇH class (0.936), meaning predictions of ÇH are almost always correct. Conversely, the model's recall for ÇH was 0.729, indicating that a considerable number of ÇH instances were misclassified as AH. The model exhibited high recall for the AH class (0.724), correctly identifying the vast majority of AH instances. However, the model's precision for the AH class was low (0.324), meaning many predictions of AH were actually ÇH instances. This asymmetry can be attributed to the significant class imbalance and the model's tendency to over-predict the majority class (ÇH). Nevertheless, the F1-scores indicate that the LLM maintained some degree of discriminative power, achieving an F1-score of 0.448 for the AH class and 0.820 for the ÇH class. The severity classifier therefore performed moderately well, though additional work would be needed to address the class imbalance and improve the model's ability to distinguish between the two severity levels.

Table 4.16: *Severity classification metrics (AH vs. ÇH).*

Class	Precision	Recall	F1-score	Support
AH	0.324	0.724	0.448	540
ÇH	0.936	0.729	0.820	3006
Accuracy				0.728
Macro avg	0.630	0.726	0.634	3546
Weighted avg	0.843	0.728	0.763	3546

4.3 Comparative Analysis

This section presents an aggregate comparison of all three classification methods evaluated in this thesis: fine-tuned BERT, TF-IDF with traditional machine learning, and zero-shot LLM classification.

4.3.1 Stage 1: Sentence Classification Comparison

Table 4.17 summarises the performance of all three methods across the three binary sentence classification tasks. Fine-tuned BERT clearly outperformed both baseline approaches across all tasks and metrics.

Table 4.17: *Comparison of Stage 1 sentence classification results across all methods.*

	BERT		**TF-IDF + SVM**		**LLM**	
Task	Accuracy	Macro F1	Accuracy	Macro F1	Accuracy	Macro F1
H vs. NH	**0.9752**	**0.9599**	0.9699	0.9521	0.7985	0.7180
Y vs. NY	**0.9515**	**0.9128**	0.9313	0.8768	0.5014	0.4870
B vs. NB	**0.9520**	**0.8855**	0.9308	0.8449	0.4345	0.3850
Average	**0.9596**	**0.9194**	0.9440	0.8913	0.5781	0.5300

4.3.2 Stage 2: Damage Severity Classification Comparison

Table 4.18 presents the performance comparison for the damage severity classification task (AH vs. ÇH). BERT again exhibited higher performance than both TF-IDF and the LLM baseline.

Table 4.18: *Comparison of Stage 2 damage severity classification results across all methods.*

Method	Accuracy	Macro F1
BERT	**0.9042**	**0.8248**
TF-IDF + LogReg	0.8761	0.7733
LLM (Zero-shot)	0.7280	0.6340

4.3.3 Summary of Results

Results from the experiments show clearly, a hierarchical performance level exists among the evaluation methods and their performance levels. The BERT model was fine-tuned to provide the best performance with respect to accuracy and macro F1 values for each of the classification tasks. In particular, the BERT model achieved the best performance on the damage detection task (H vs. NH) with an accuracy value of 97.52%. The results of these evaluations confirm that the ability of BERT to represent contextual semantic relationships and its flexibility to be applied to highly variable Turkish social media texts have been major contributing factors to the success of BERT based methods.

TF-IDF baseline models using traditional machine learning classifiers (SVM linear classifier for sentence classification and logistic regression for severity) had comparable performance to fine-tuned BERT model for those tasks that involved lexical characteristics (damage detection). However, when a greater contextual understanding was required (assistance need detection and information classification) than what TF-IDF was capable of providing, the performance gap between BERT and TF-IDF increased.

Performance by zero-shot LLM approach, which does not require task-specific training data, were lower than both BERT and TF-IDF across all classification tasks, particularly for assistance (Y) and information (B) classification where accuracy rates

were near-random at 50.14% and 43.45%, respectively. The data suggests that prompt-based classification is limited for fine-grained operational categories in disaster response situations and indicates the benefits of domain-specific training data.

These results provided the basis for selecting BERT based classifiers for deployment in the production system. In addition to raw accuracy metrics, several factors contributed to this decision. First, BERT's ability to provide contextual embeddings were important in handling the linguistic variability found in Turkish social media texts including informal grammar, spelling variations and morphological complexity of Turkish. Second, although TF-IDF achieved competitive accuracy on lexically distinct tasks such as damage detection, its accuracy decreased with increasing levels of semantics complexity for classifying complex categories requiring contextual comprehension. Third, the zero-shot LLM approach failed to achieve operational-level accuracy for fine grained disaster response categories despite no training data being required. Fourth, although BERT has higher inference latency than TF-IDF, it remains acceptable for near-real-time classification when deployed on cloud infrastructure with sufficient resource allocation. Fifth, the combination of these considerations validates the selection of BERT as the foundation for the operational system described in the following chapter.

5. SYSTEM DESIGN AND EXECUTION

The overall system structure, including the complete system architecture and execution, for automatically assessing earthquake damage and categorizing the need for assistance is outlined in this chapter based on the empirical findings reported in Chapter 4. Since they achieved superior performance—97.52 percent accuracy for damage detection (H vs. NH), 95.15 percent for assistance need detection (Y vs. NY), 95.20 percent for information detection (B vs. NB), and 90.42 percent for damage severity classification (AH vs. ÇH)—the BERT-based classifiers were chosen for deployment. Because of these high levels of precision, in conjunction with BERT's ability to handle linguistic variability, BERT is the best option for operational deployment even though it needs more computation than TF-IDF-based alternatives.

In this chapter, the authors describe how the BERT-based models were incorporated into a fully functional system that can scale to run continually in near real time. On an ongoing basis, the system collects Ekşi Sözlük posts, processes them through the preprocessing pipeline described previously, and classifies each sentence through a multi-stage transformer-based inference process hosted on cloud infrastructure.

5.1 Overall System Structure

At a general level, the system design consists of four major phases:

1. Monitor and collect Ekşi Sözlük postings on an ongoing basis,
2. Normalize and preprocess the text at the sentence level,
3. Classify each sentence through a multi-stage transformer-based classifier,
4. Host the system on cloud-based infrastructure with an API to enable interaction with other applications.

Figure 5.1 illustrates the overall system architecture and data flow.

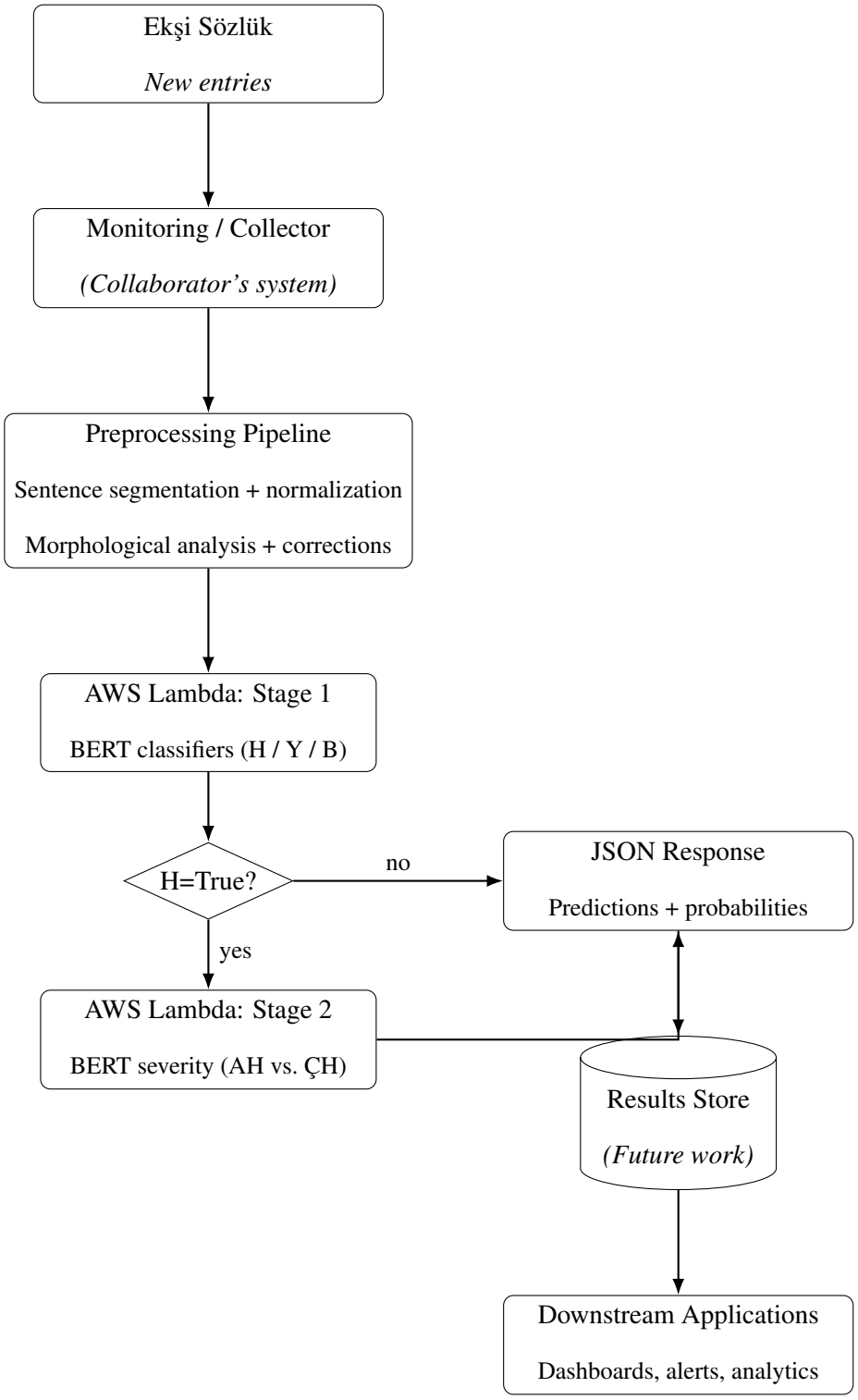

Figure 5.1: *End-to-end system architecture. Stage 1 classifies content type (H, Y, B) and Stage 2 conditionally determines damage severity (AH vs. ÇH).*

5.2 Data Collection and Preprocessing Pipeline

The purpose of establishing the data collection and preprocessing pipeline was to ensure that the data fed into the models were as consistent as feasible (in terms of distribution and format) to the data processed during the off-line dataset development and the data processed during the on-line system operation.

To achieve this objective, new data collected from Ekşi Sözlük are sent to the exact same data-preprocessing pipeline described in Section 3.3, thereby providing consistency between the data utilized for developing the off-line datasets and the data processed in the on-line system.

Therefore, the preprocessing pipeline is the same whether it is employed to develop the off-line dataset or to develop the on-line system.

5.2.1 Extraction and Segmentation of Raw Text and Sentences

Raw text is extracted from postings obtained from Ekşi Sözlük and segmented into separate sentences. Because all downstream classifiers function at the sentence level, the segmentation of sentences removes any uncertainty associated with lengthy, multi-topic postings and enables the precise classification of each individual declaration concerning either damage or assistance.

5.2.2 Morphological Analysis and Standardization

Each sentence is subjected to morphological analysis to produce tokens, obtain root forms, and determine the grammatical characteristics of the tokens. Tokens that cannot be produced during the first pass are logged and processed through an iterative refinement process consisting of adjustments to the capitalization, validation against dictionaries, removal of ASCII characters, and verification of spelling. Following manual inspection and validation of a token, the validated version is reintroduced into the corpus resulting in

a standardized and linguistically consistent representation of the sentence.

Therefore, the preprocessing pipeline ensures that input streams available at the inference time have similar distributions and structures to those of the data utilized to develop the models.

5.3 Design of the Multi-Stage Classification Pipeline

The logic for inference of the system is established on a two-stage classification pipeline, which is presented as a cloud-based service.

5.3.1 Stage 1: Determination of the Type of Content

At Stage 1, each normalized sentence is independently assessed using three binary BERT-based classifiers trained utilizing the Deprem Tam Data Annotation Scheme:

- **H vs. NH** = Physical damage identification,
- **Y vs. NY** = Identification of requests or offers of assistance,
- **B vs. NB** = Identification of informative content.

A probability value for the positive category is generated as output by each classifier. A threshold of 0.5 is applied to these probability values to produce a binary determination. The outcome of this stage is a structured prediction illustrating whether the sentence is categorized as containing damage-related, assistance-related and/or informative content.

As previously mentioned, this design takes into account the multidimensional aspect of disaster-related dialogue, where one sentence may indicate damage, request assistance and provide context-specific information.

5.3.2 Stage 2: Damage Severity Estimation

Stage 2 is conducted only when the determination from Stage 1 identifies the sentence as damage-related (H = True). Stage 2 will then evaluate the same sentence

employing a model trained for estimating damage severity using the secondary annotation scheme (AH vs. ÇH).

The model determines if the damage mentioned in the sentence is moderate (**AH**) or severe (**ÇH**). To avoid excessive computation and to establish a logical dependency between the detection of content and the estimation of damage severity, the damage severity estimator is executed only for those sentences identified as damage-related.

5.4 Deployment Using AWS Lambda

All classifiers in this study are containerized, packaged, and deployed through AWS Lambda as an inference service. The implementation utilizes a Python-based inference service that loads four fine-tuned BERT models:

- `tamDepremBert_H` – Damage Detection Classifier (H vs. NH)
- `tamDepremBert_Y` – Classifier for Assistance Need (Y vs. NY)
- `tamDepremBert_B` – Classifier for Information (B vs. NB)
- `bert-hasarli-final` – Classifier for Damage Severity (AH vs. ÇH)

5.4.1 Implementation of Inference Service

At its core, the inference service takes the form of a Lambda handler function written in `app.py`. When the function first starts up, it pulls in all the necessary models through the HuggingFace Transformers library. Each of these models comes with its weights saved in `safetensors` format, alongside the tokeniser configuration files they need to operate.

The function exposes just one API endpoint. It takes in a JSON payload with a sentence and hands back the classification results. The following operations occur when a request comes through to the Lambda handler:

1. First, it pulls the text field out of the incoming JSON payload

2. Next, it runs the input sentence through the tokenisers that belong to the Stage 1 classifiers—capping the sequence at 256 tokens, with truncation and padding as needed

3. The tokenised input then goes through forward passes in each of the three binary classifiers from Stage 1

4. Softmax turns the output into a probability distribution, and the function identifies the positive-class probability for each classifier

5. These probabilities are converted to a binary format (i.e., values above 0.5 are positive)

6. When the damage prediction (H) identifies positive damage, the function uses the damage severity model (`bert-hasarli-final`) to identify whether the severity falls into AH (mild) or ÇH (severe)

7. Finally, the function formats all relevant data—the original input text, Stage 1 predictions, and Stage 2 severity information—into a JSON object that is returned to the caller

Note that the models are loaded once when Lambda starts and remain in memory throughout the life of the function, because they reside at `/opt/models` and are read-only. This reduces the overall time of inference as much as possible.

5.4.2 Containerisation

The Lambda function runs in a Docker container utilising the public AWS Python 3.11 Lambda base image (`public.ecr.aws/lambda/python:3.11`). The Dockerfile has the following components:

- **Base dependencies**: pip installs NumPy version 1.26.4 and Transformers version 4.57.3
- **PyTorch CPU runtime**: Provided by the official PyTorch CPU wheel index, which reduces the overall container size
- **Model weights**: Copies the weights for the four trained BERT models and the tokeniser configurations into `/opt/models`
- **Handler script**: Defines the inference logic in the Lambda task root directory

The two main steps to create the container are to install the dependencies and copy the models into the container. After the local image is created, it is uploaded to AWS ECR (Elastic Container Registry), allowing Lambda to download it when deployed. Containers were a good fit for this application. Containers allow full control over the dependencies available, and the behaviour of the code remains the same whether the code is running in development, testing, or production environments. One of the advantages of containers is that they can support models larger than the typical 250 MB limit that restricts the size of deployment packages for Lambda functions.

5.4.3 Test Interface

A web-based frontend was developed to help with testing and demonstrations of the classifier using Next.js and React. The frontend, called *deprem-ui*, provides a simple way for users to enter Turkish text for classification and to view the classifications that the classifiers produce.

deprem-ui is a single-page application that includes the following features:

- **Text Input Box**: The user enters the Turkish text to classify into the text input box
- **Submit Button**: When the user clicks the submit button, the text is submitted to the Lambda function through an API proxy

- **Results Panel**: The JSON response from the Lambda function is displayed in the results panel, including the Stage 1 predictions (H, Y, B) along with their respective probability scores and the Stage 2 severity classification (AH/ÇH) if applicable

deprem-ui is hosted on Vercel and is publicly accessible. The frontend provided a fast way for developers to iterate on the development process and serves as a demonstration of the classifier's ability to classify text without having to directly interact with the API.

5.4.4 Example API Response

Figure 5.2 displays an example JSON response received from the Lambda inference service. The example input describes a cat that has been trapped in a collapsed building since the 11th day post-earthquake. It indicates the location of the cat and asks for assistance from a firefighter or a crane. Evaluating the performance of the classifiers: Stage 1 identified the damage-related information (H = true, 0.997 probability) and the request for assistance (Y = true, 0.995 probability), but did not find that it was informational (B = false). Since there was evidence of damage, Stage 2 was invoked and the severity of the damage was classified as ÇH (severe damage) with 99.5% confidence. This classification is reasonable based on the text describing a collapsed five-story building.

```
{
  "input": "11. gun, 5. kat yikik bir evde caresizce
            bekleyen bir kedi. Konum: Hatay Cebrail
            Mahallesi... Lutfen destek.",
  "stage1": {
    "predicted": { "H": true, "B": false, "Y": true },
    "probabilities": {
      "H": 0.9969,
      "B": 0.0001,
      "Y": 0.9949
    }
  },
  "stage2": {
    "triggered": true,
    "bert_hasarli_final": {
      "pred_label": "CH",
      "probs": { "AH": 0.0038, "CH": 0.9951 }
    }
  }
}
```


Figure 5.2: *A sample JSON response from the Lambda inference service. The input text talks about a cat trapped in a collapsed building, along with a request for rescue help. Stage 1 correctly flags damage (H) and assistance need (Y), and Stage 2 classifies the damage as ÇH (severe).*

5.5 Overarching View of Full System Operation

The structure of the system we built was a complete stack to analyse earthquake damage in real-time. Here is a top-down view of how the entire system operates, from start to finish:

1. Entries to Ekşi Sözlük are constantly being generated. A collaborating colleague has created a collector which continually collects new entries and runs in the background; however, since the collector was not created as part of this thesis, it will not be discussed further.

2. Once the entries are collected, they are passed through the entry processing pipeline. This pipeline normalises the text and breaks it into individual sentences.

3. Each of these individual sentences is then passed to the Lambda inference API in a JSON formatted payload.

4. Each input sentence to the Stage 1 classifier produces a binary classification for each of the three categories: Damage, Assistance Needed, and Informational Content.

5. If the Stage 1 classifier finds that the input sentence relates to damage (H = True), the Stage 2 classifier is called to determine the level of damage (AH for mild, ÇH for severe).

6. The classifications produced from both classifiers are returned in a structured JSON format and can be saved to a database or passed to other applications that use the classifications for alerting, notifications, or visualisation.

This architecture allows us to monitor social media conversations in near real-time to identify earthquake-related information. Additionally, because the output is structured and machine-readable, it can be used to provide input to decision-making systems, trigger

automatic notifications, or allow humans to interact with disaster response workflows in the loop. Although storing the classifications to a database is part of the remaining work, it is a necessary step to take advantage of the classifications as input to downstream applications.

5.6 Justification for Design Choices

A large number of the design choices were made based upon the needs of actual disaster response systems, including:

- **Modularity**: Each classification task (H, Y, B, Severity) has its own model. Thus, if one of the models is updated or retrained, it will not affect the performance of the other parts of the system.
- **Scalability**: Horizontal scaling of the system is handled directly by AWS Lambda. Regardless of the amount of social media activity occurring immediately after an earthquake, the system will continue to operate.
- **Efficiency**: The Stage 2 severity classifier only runs when something has already been flagged as damage-related (H = True). This avoids unnecessary computation on entries that do not require that additional classification step.
- **Reproducibility**: With container-based deployment, the system behaves the same way whether it is running in development, testing, or production.
- **Model Selection**: The experiments showed that BERT-based models achieved the best balance between accuracy and inference speed, making them the clear choice over TF-IDF and the LLM approaches for real-time classification.

Pulling together efficient preprocessing, transformer-based inference, and cloud-native deployment gave the system a shape that actually makes the research objectives from this thesis achievable in practice.

6. CONCLUSION AND FUTURE WORK

This thesis set out to build a framework for automatically assessing earthquake damage and classifying assistance needs based on social media data from Ekşi Sözlük. The work demonstrated that Turkish social media posts contain valuable information for disaster response, and that transformer-based models can pull structured information out of what is otherwise messy, unstructured text. This chapter goes over what the research contributed, what was learned along the way, where the limitations lie, and where things could go from here.

6.1 Summary of Research Contributions

This dissertation contributes to both disaster informatics and NLP for Turkish:

1. **Novel Turkish Earthquake Dataset**: A total of 24,900 entries were collected from Ekşi Sözlük across several earthquake events in Turkey. Of those, 18,628 were manually labelled at the sentence level as H/Y/B for content type and AH/ÇH for damage severity. Thus, this dataset could be useful for any future researchers in Turkish disaster informatics.

2. **Multi-Stage Classification Pipeline**: The approach presented here is composed of two stages. In Stage 1, the system classifies content types (damage, assistance needed, information). Only if the entry is classified as damage in Stage 1 will Stage 2 be activated to classify the entry based on severity. By structuring the pipeline in such a way, we avoid unnecessary computations and keep the relationships between tasks clear.

3. **Comparative Evaluation**: We evaluated three different classification methods head-to-head: fine-tuned BERT, TF-IDF paired with traditional machine learning,

and zero-shot LLM classification. Comparing these methods allowed us to show empirically which performs best for disaster response applications.

4. **Deployable System**: We built a production-ready inference service based on AWS Lambda and containerised BERT models. The fact that transformer-based classifiers can now be used for real-time disaster monitoring—both theoretically and practically—is demonstrated by this example.

6.2 Experimental Results

We found the following results from our experiments:

- **Fine-Tuned BERT Performs Best**: On average, fine-tuned BERT performed better than the other two classification methods across all classification tasks. Specifically, fine-tuned BERT achieved an accuracy of 97.52% for the damage detection task (H vs. NH). TF-IDF performed at an accuracy of 96.99%, and the LLM approach achieved an accuracy of 79.85%. These numbers clearly indicate that fine-tuning BERT on Turkish social media text is recommended, especially in a specific domain like this.

- **Limitations of the LLM Method**: The zero-shot LLM method performed poorly on fine-grained categories, producing essentially random accuracy for assistance (50.14%) and information (43.45%) categories. These results suggest that prompt-based classification without task-specific training may not be suitable for operational disaster response categories. It should be noted that `gpt-4o-mini` was used due to budget constraints; larger models or few-shot prompting with contextual examples could yield different results.

- **Success of the Two-Stage Approach**: Activating the second severity classification stage only after the first stage determined that the content type is damage (i.e., H

= True) worked well, achieving 90.42% accuracy on the AH vs. ÇH classification. Additionally, activating the second stage only after determining that the content type is damage avoids wasted computation on entries that are unrelated to damage.

6.3 Limitations of the Dataset

There are several limitations of the dataset worth noting:

- **Class Imbalance**: The dataset has a notable class imbalance, especially for the damage severity task. ÇH (severe damage) samples outnumber AH (mild damage) samples by roughly 5.5:1 (3,006 vs. 540). This imbalance makes training and evaluation harder. Class weights were applied during training to try to address the problem, but the minority class (AH) still proved difficult to classify reliably.

- **A Single Platform**: Everything in the corpus comes from one platform—Ekşi Sözlük, a Turkish collaborative dictionary site that attracts a particular kind of user with a particular writing style. Models trained on this data probably will not behave the same way on other platforms like Twitter or Instagram, since those differ in character limits, user demographics, and how people express themselves.

- **Scope of Annotation**: Time constraints and limited annotator availability meant that only 18,628 of the 24,900 collected entries got fully annotated. The rest could serve as additional training data if someone annotates them in a future study.

6.4 Practical Implications for Disaster Response

The system built for this thesis has some real practical implications for emergency management:

- **Compatibility with Emergency Management Systems**: Because the classification system outputs structured JSON, it can slot into existing emergency management

workflows without too much trouble. The damage severity classification (AH vs. ÇH) can help responders figure out where to direct resources first—prioritising areas with reports of severe structural damage.

- **Automatic Generation of Alerts**: The BERT classifiers are precise enough to support automatic alert generation without flooding responders with false positives. Posts that get classified as severe damage (ÇH) with high confidence can trigger immediate notifications to the right teams.

- **Structured Decision-Making Information**: Taking unstructured social media text and turning it into structured, machine-readable classifications gives emergency officials something they can actually act on. The multi-dimensional output—covering damage, assistance needs, and general information—lets officials filter and aggregate data according to whatever their operational priorities happen to be.

6.5 Lessons Learned

A few lessons came out of this research that might be useful for future work in Turkish NLP and disaster informatics:

- **Turkish-Specific Preprocessing**: Turkish is morphologically complex, and that means specialised preprocessing is not optional. The pipeline developed here—using NLPToolkit libraries for morphological analysis, deasciification, and spell checking—turned out to be essential for normalising messy social media text. Without these preprocessing steps, model performance would almost certainly have been much worse.

- **Sentence-Level Classification**: Classifying individual sentences rather than whole Ekşi Sözlük entries worked better for disaster response purposes. A single post often

covers multiple topics—it might describe building damage, ask for help, and share general information all at once. Working at the sentence level captures that detail.

- **Domain-Specific Fine-Tuning**: The significant performance gap between fine-tuned BERT and zero-shot LLM classification underscores the importance of domain-specific training data. Pre-trained language models perform well in general, but achieving operational-level accuracy in a specialised area like disaster response requires task-specific fine-tuning.

6.6 Inference Performance and Scalability

BERT-based models deliver noticeably better classification performance than TF-IDF, but their computational costs are something to think about when deploying in real time:

- **Inference Latency**: Running inference with BERT takes substantially more computation than TF-IDF-based classification, which means longer latency. The current AWS Lambda setup works, but it can run into delays from cold starts. A cold start happens when the Lambda container first initialises and the models have to be loaded into memory. Formal latency benchmarking was not conducted as part of this thesis; however, informal testing indicated response times of approximately 2–4 seconds per classification request under warm-start conditions. Systematic measurement of cold-start latency, throughput under load, and performance during traffic spikes remains an area for future work.

- **Scalability During Disaster Response**: The system's ability to scale up when disaster strikes—and handle the surge in social media traffic that follows an earthquake—is critical. AWS Lambda will need to process far more requests than it normally sees. While Lambda does scale automatically based on invocation count, the first spike in

traffic can still suffer from higher latency as new instances spin up.

- **Methods of Reducing Inference Time**: There are a few ways to speed things up when running inference in production:
  - When disaster-related traffic spikes, bumping up the memory allocated to the AWS Lambda function can make inference faster
  - AWS Lambda Provisioned Concurrency keeps a pool of warm instances ready, avoiding the wait for cold starts
  - Model quantisation or distillation shrinks the model size and speeds up inference without hurting accuracy
  - A hybrid approach could use TF-IDF to quickly filter out obvious cases, saving BERT for the ones that need closer analysis
- **Trade-offs Between Accuracy and Speed**: Choosing between BERT and TF-IDF comes down to a trade-off: classification accuracy versus inference speed. For time-sensitive situations, a tiered approach might work well—let simpler models handle the bulk of straightforward cases and reserve BERT for the ambiguous ones.

6.7 Future Work—Unimplemented Parts of the System Architecture

Some parts of the overall system architecture shown in Figure 5.1 still need to be built:

- **Real-Time Collection of New Ekşi Sözlük Entries**: A colleague who collaborated on this project built a separate system for collecting new Ekşi Sözlük entries in real time, so this thesis did not include that component.
- **Persistent Storage of Classified Posts**: Right now, the system hands back classification results as JSON but does not store them anywhere permanent. Hooking up

AWS DynamoDB or something similar would make it possible to analyse historical trends and track how damage reporting changes over time.

- **Visualisation Interface for Emergency Managers**: A live dashboard that allows emergency managers to view classification results and watch how damage reports evolve—there is no tool like this available at present.

6.8 Future Work

The following are some key areas that future research could address:

1. *Dealing with Class Imbalance*: There are a number of ways in which researchers can attempt to deal with imbalance in classes (particularly the under-represented AH class for mild damage) using oversampling, data augmentation, or cost-sensitive learning to improve classifier performance.

2. *Multi-Platform Expansion*: To increase coverage and reliability of the data collected, it would be beneficial to expand the system to include additional social media platforms such as Twitter and Instagram.

3. *Developing the Real-Time Dashboard*: The development of a visualisation interface for emergency management officials—one that displays classified posts on a map, shows temporal trends, and allows for filtering of posts based on damage severity and assistance needs—is essential for effective use of this system.

4. *Optimising Models*: There are several model optimisation techniques (knowledge distillation, quantisation, and pruning) that can be researched and implemented to decrease the latency associated with inference while maintaining the classifiers' accuracy levels.

5. *Multi-Label Classification*: Changing the classification task to a multi-label prediction task can assist in capturing posts that have multiple categories (e.g., damage reports and assistance requests).

Overall, this dissertation illustrates the value of social media content in providing actionable information during earthquake disasters; that transformer-based models can successfully extract this information for disaster response applications; and that the system developed here represents an initial foundation for real-time monitoring and classification of Turkish social media content, with many avenues for expansion and deployment.

REFERENCES

[1] L. Palen and S. B. Liu, "Citizen communications in crisis: Anticipating a future of ict-supported public participation," *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 727–736, 2007.

[2] C. Reuter and M. A. Kaufhold, "Fifteen years of social media in emergencies: A retrospective review and future directions for crisis informatics," *Journal of Contingencies and Crisis Management*, vol. 26, no. 1, pp. 41–57, 2018.

[3] J. B. Houston, J. Hawthorne, M. F. Perreault, E. H. Park, M. Goldstein Hode, M. R. Halliwell, S. E. Turner McGowen, R. Davis, S. Vaid, J. A. McElderry, and S. A. Griffith, "Social media and disasters: A functional framework for social media use in disaster planning, response, and research," *Disasters*, vol. 39, no. 1, pp. 1–22, 2015.

[4] S. R. Veil, T. Buehner, and M. J. Palenchar, "A work-in-progress literature review: Incorporating social media in risk and crisis communication," *Journal of Contingencies and Crisis Management*, vol. 19, no. 2, pp. 110–122, 2011.

[5] T. Simon, A. Goldberg, and B. Adini, "Socializing in emergencies: A review of the use of social media in emergency situations," *International Journal of Information Management*, vol. 35, no. 5, pp. 609–619, 2015.

[6] B. R. Lindsay, "Social media and disasters: Current uses, future options, and policy considerations," Tech. Rep. R41987, Congressional Research Service, 2011.

[7] D. E. Alexander, "Social media in disaster risk reduction and crisis management," *Science and Engineering Ethics*, vol. 20, no. 3, pp. 717–733, 2014.

[8] A. L. Hughes and L. Palen, "The evolving role of the public information officer: An examination of social media in emergency management," *Journal of Homeland Security and Emergency Management*, vol. 9, no. 1, 2012.

[9] O. Okolloh, "Ushahidi, or "testimony": Web 2.0 tools for crowdsourcing crisis information," *Participatory Learning and Action*, vol. 59, no. 1, pp. 65–70, 2009.

[10] M. Zook, M. Graham, T. Shelton, and S. Gorman, "Volunteered geographic information and crowdsourcing disaster relief: A case study of the haitian earthquake," *World Medical & Health Policy*, vol. 2, no. 2, pp. 7–33, 2010.

[11] K. Starbird and L. Palen, ""voluntweeters": Self-organizing by digital volunteers in times of crisis," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 1071–1080, ACM, 2011.

[12] P. Meier, *Digital Humanitarians: How Big Data Is Changing the Face of Humanitarian Response*. CRC Press, 2015.

[13] R. I. Ogie, R. J. Clarke, H. Forehead, and P. Perez, "Crowdsourced social media data for disaster management: Lessons from the petajakarta.org project," *Computers, Environment and Urban Systems*, vol. 73, pp. 108–117, 2019.

[14] P. S. Earle, M. Guy, R. Buckmaster, C. Ostrum, S. Horvath, and A. Vaughan, "Omg earthquake! can twitter improve earthquake response?," *Seismological Research Letters*, vol. 81, no. 2, pp. 246–251, 2010.

[15] T. Sakaki, M. Okazaki, and Y. Matsuo, "Earthquake shakes twitter users: Real-time event detection by social sensors," in *Proceedings of the 19th International Conference on World Wide Web*, pp. 851–860, ACM, 2010.

[16] S. Vieweg, A. L. Hughes, K. Starbird, and L. Palen, "Microblogging during two natural hazards events: What twitter may contribute to situational awareness," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 1079–1088, ACM, 2010.

[17] J. Fohringer, D. Dransch, H. Kreibich, and K. Schröter, "Social media as an information source for rapid flood inundation mapping," *Natural Hazards and Earth System Sciences*, vol. 15, no. 12, pp. 2725–2738, 2015.

[18] Y. Kryvasheyeu, H. Chen, N. Obradovich, E. Moro, P. Van Hentenryck, J. Fowler, and M. Cebrian, "Rapid assessment of disaster damage using social media activity," *Science Advances*, vol. 2, no. 3, p. e1500779, 2016.

[19] M. Imran, C. Castillo, J. Lucas, P. Meier, and S. Vieweg, "Aidr: Artificial intelligence for disaster response," in *Proceedings of the 23rd International Conference on World Wide Web Companion*, pp. 159–162, ACM, 2014.

[20] Z. Ashktorab, C. Brown, M. Nandi, and A. Culotta, "Tweedr: Mining twitter to inform disaster response," in *Proceedings of the 11th International Conference on Information Systems for Crisis Response and Management (ISCRAM)*, 2014.

[21] A. Olteanu, S. Vieweg, and C. Castillo, "What to expect when the unexpected happens: Social media communications across crises," in *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing*, pp. 994–1009, ACM, 2015.

[22] J. Ray Chowdhury, C. Caragea, and D. Caragea, "Cross-lingual disaster-related multi-label tweet classification with manifold mixup," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: Student Research Workshop*, pp. 292–298, ACL, 2020.

[23] V. Linardos, M. Drakaki, P. Tzionas, and Y. L. Karnavas, "Machine learning in disaster management: Recent developments in methods and applications," *Machine Learning and Knowledge Extraction*, vol. 4, no. 2, pp. 446–473, 2022.

[24] V. Linardos, M. Drakaki, P. Tzionas, and Y. L. Karnavas, "Utilizing llms and ml algorithms in disaster-related social media content," *GeoHazards*, vol. 6, no. 3, p. 33, 2025.

[25] A. Acar and Y. Muraki, "Twitter for crisis communication: Lessons learned from japan's tsunami disaster," *International Journal of Web Based Communities*, vol. 7, no. 3, pp. 392–402, 2011.

[26] B. D. M. Peary, R. Shaw, and Y. Takeuchi, "Utilization of social media in the east japan earthquake and tsunami and its effectiveness," *Journal of Natural Disaster Science*, vol. 34, no. 1, pp. 3–18, 2012.

[27] D. Ertuncay, L. Cataldi, and G. Costa, "Web-based macroseismic intensity study in turkey – entries on ekşi sözlük," *Geoscience Communication*, vol. 4, no. 1, pp. 69–81, 2021.

[28] M. Mendoza, B. Poblete, and C. Castillo, "Twitter under crisis: Can we trust what we rt?," in *Proceedings of the 1st Workshop on Social Media Analytics (SOMA)*, pp. 71–79, ACM, 2010.

[29] O. Oh, M. Agrawal, and H. R. Rao, "Community intelligence and social media services: A rumor theoretic analysis of tweets during social crises," *MIS Quarterly*, vol. 37, no. 2, pp. 407–426, 2013.

[30] J. A. Van Dijk, "Digital divide research, achievements and shortcomings," *Poetics*, vol. 34, no. 4–5, pp. 221–235, 2006.

[31] T. Acıkara, B. Xia, T. Yigitcanlar, and C. Hon, "Contribution of social media analytics to disaster response effectiveness: A systematic review of the literature," *Sustainability*, vol. 15, no. 11, p. 8860, 2023.